

tramdag documentation

Contents

tramdag — Interpretable Neural Causal Models (TRAM-DAGs) in PyTorch	3
Start here	3
Install	3
Documentation	3
Experiments	4
Layout	4
Use of AI tools	4
Citation	4
The model	5
The transformation function	5
The components	5
The components in code	6
Ordinal nodes	7
What the model cannot do	7
Notation	7
Symbols	7
The model's symbols	8
Reading a fitted TRAM-DAG	8
Which term owns which edge	8
A linear shift is a log-odds ratio	9
Ordinal nodes: cutpoints and the interventional PMF	9
Continuous nodes: density and one fitted shift	9
The price of a complex intercept	10
An effect that varies with a covariate	10
Fitting a TRAM-DAG	10
One module, one sub-model per node	10
The likelihood	10
Path A: stochastic optimization with fit	11
Path B: classical optimization with fit_classical	12
Memory and disk	12
Per-observation scores and the effect-modifier scan	13
The effect-modifier scan	13
Varying-coefficient treatment effects: VC(*modifiers, t=, penalty=, center=)	13
Why not a joint CS over treatment and modifiers	13
Semantics	14
Propensity centering: center="col"	14
Validation	15

Paper replication: protocol, hyperparameters and results, per experiment	15
What is common to all eight	15
Triangle, continuous (triangle.py): paper Sec. 6.1, App. C.3	17
Triangle, mixed (triangle_mixed.py): paper Sec. 6.2, App. C.4, App. B	18
VACA / CNF benchmark (vaca.py): paper Sec. 5.1–5.2, App. C.1	19
CAREFL benchmark (carefl.py): paper Sec. 5.3, App. C.2	20
D4: the marginal start, measured per variant	21
Runtime and the CI deviations	22
Repository choices the paper does not state	23
Code map — every module of src/tramdag/ — the public names and the private plumbing	23
spec.py — declare the model	23
transforms.py — the monotone map h and the ordinal transform	24
flow.py — the model	25
modules.py — the term modules	26
nodes.py — the node model	27
fitting.py — FitMixin	27
readouts.py — ReadoutsMixin	27
scores.py — effect-modifier detection	27
callbacks.py — the shipped fit callbacks	28
plots.py — the figures (matplotlib optional: tramdag[plots])	28
What is <i>not</i> in the package	28
Where every training hyperparameter lives	28
Upstream candidates for zuko	29
1. Analytic call_and_ladj for BernsteinTransform	30
2. Learnable/linear tails for MonotonicRQSTransform	30
3. Public inverse of _constrain_theta	30
4. Docstring fix: the Bernstein θ -shape off-by-one	30
5. Logistic in zuko.distributions	30
Anti-candidates (look upstreamable, are not)	30
Additive vs joint complex intercept — interpreting per-parent effects	31
The data	31
Fit both models	32
intercept_contributions — the exact, centered decomposition	33
The structural difference: separable curves vs an entangled cloud	33
Takeaway	34
Classical fitting of all-ls TRAM-DAGs (fit_classical)	34
0. The zeroth example — logistic regression on MASS::birthwt	35
1. Continuous case	38
1b. A bimodal DAG — the demo data	42
2. Ordinal case	45
3. Warm-start handoff: classical fit $\rightarrow$ further training	46
TRAM-DAG: one causal model, all three rungs	47
1. The benchmark process	48
2. The spec is the DAG, and one fit	49
3. Rung 1: the observational fit	51
4. Rung 2: interventions	52
5. Rung 3: individual counterfactuals	52
What this notebook checked	53
Training strategies: driving fit from the API side	54

1. What fit records without any callback	55
2. Plain Adam	56
3. Two phases	56
4. EarlyStopping: keep the best weights	57
5. EarlyStopping(patience=): stop as well as restore	57
6. Per-node rates: per_node_adam with PerNodeEarlyStopping	58
7. Writing your own	59
8. The scoreboard: a poor man’s runtime comparison	60
Which one, when	61

Heterogeneous treatment effects: the VC term 62

1. A DGP with a known effect function	62
2. Which covariates modify the effect? (effect_modifier_scan)	63
3. The spec: prognostic part and effect head are separate	64
4. Reading the effect out	65
5. The read-out is an identity, not a summary	65
6. Confounding: why center= exists	66
What to take away	67

tramdag — Interpretable Neural Causal Models (TRAM-DAGs) in PyTorch

TRAM-DAGs model each variable of a structural causal model with a monotone normalizing flow, the statistician’s *transformation model*. The whole model is one triangular normalizing flow from iid standard-logistic latents to the observed variables, and its triangular structure is exactly your causal DAG. One fit on observational data answers all three rungs of Pearl’s causal hierarchy: observational, interventional (the do-operator) and counterfactual (abduction). The model keeps **interpretable effects**: every linear-shift coefficient is a log-odds ratio, as in a classical proportional-odds model.

Beate Sick & Oliver Dürr, *Interpretable Neural Causal Models with TRAM-DAGs*, CLeaR 2025 (arXiv:2503.16206). This repository is the reference implementation, in PyTorch on zuko. It replicates the paper’s experiments under continuous integration.

Start here

The demo notebook is the introduction. It fits the paper’s bimodal benchmark, walks the three rungs, and checks every answer against analytic ground truth. It runs on CPU in a few minutes, in Colab or locally.

Install

The package is on PyPI as tramdag; `uv add tramdag` or `pip install tramdag` installs it, and the `plots` extra (`tramdag[plots]`) adds matplotlib for the figures. The PyPI release lags main; for the current state install from git (`tramdag @ git+https://github.com/tensorchiefs/tramdag.git@main`, pinned to a commit for reproducibility). A clone with `uv sync` gives the development setup with tests and experiments.

Documentation

Guides, executed notebooks and the API reference are published at the documentation site. Each subject has one guide, and its worked example is one notebook.

Subject	Guide	Example
The model, its notation, what it cannot do	docs/model.md, docs/notation.md	demo
Reading a fitted model: coefficients, curves, interventional distributions	docs/interpretation.md	classical fitting
Fitting: the likelihood, the Adam loop and its callbacks, the classical fit, a runtime comparison of the recipes	docs/fitting.md	training strategies
Complex intercepts, joint versus additive		additive vs joint intercepts
Treatment effects that vary with covariates	docs/varying-coefficients.md	varying coefficients
Scores and the effect-modifier scan	docs/scores.md	varying coefficients
Agreement with statsmodels and R, standard errors		classical fitting
What the tests guarantee	tests/README.md	
The API, every default and where it lives	docs/code-map.md	
How the package is built inside	docs/architecture.md	
zuko, and what tramdag would upstream	docs/zuko-upstream.md	

CONTRIBUTING.md covers setup, tests, linting and releases; notebooks/README.md how the notebooks are written and run.

Experiments

experiments/ replicates the paper's experiments against its own R code and checks them against committed ground truth in continuous integration. experiments/README.md explains the layout, the YAML variants and the frozen data; docs/paper-replication.md gives the protocol, the hyperparameters and the numbers, experiment by experiment.

Layout

- src/tramdag/ is the framework, and nothing else.
- tests/ measures it against three inline data-generating processes; it never imports research code.
- experiments/ holds the paper replications and their frozen data sets.
- notebooks/ are jupyter .py files, executed by the docs workflow.
- docs/ are the guides; the API pages render from the docstrings.

Use of AI tools

This project was developed with the help of LLM coding assistants. The authors reviewed and take responsibility for all content.

Citation

If you use tramdag, please cite the method paper:

```
@inproceedings{sick2025tramdag,
  title      = {Interpretable Neural Causal Models with TRAM-DAGs},
  author     = {Sick, Beate and D{\\"u}rr, Oliver},
  booktitle  = {Proceedings of the 4th Conference on Causal Learning and Reasoning (CLeaR)},
  series     = {Proceedings of Machine Learning Research},
  volume     = {275},
  year      = {2025},
}
```

The model

A TRAM-DAG is one triangular normalizing flow whose sparsity is a causal DAG. This page states the model in the notation of the paper and maps each part of it onto the constructors in `tramdag.spec`. For the symbols see `notation.md`; to read a model after it is fitted see `interpretation.md`; for a worked fit see the `demo notebook`.

The transformation function

The variables have a causal ordering. For each variable a TRAM-DAG fits a monotone transformation function h . The function maps the observed value to a latent scale, conditional on the parents of that variable:

$$\begin{aligned} u_1 &= h(x_1) \\ u_2 &= h(x_2 \mid x_1) \\ u_3 &= h(x_3 \mid x_1, x_2) \\ u_p &= h(x_p \mid x_1, x_2, \dots, x_{p-1}) \end{aligned}$$

This is the observed-to-latent direction. It is Eq. 2 of the paper, $F_{X|\text{pa}}(x) = F_U(h(x \mid \text{pa}))$, and in code it is `u = h(x) + shift`.

The direction matters for cost. Training evaluates h directly to score the likelihood, which is cheap. Sampling runs the inverse $x_i = h^{-1}(u_i \mid \text{pa})$. That inverse has no closed form, so bracketed bisection solves it, and sampling is the more expensive direction.

Together the h 's form one triangular flow. Each variable depends only on its causal parents $\text{pa}(x_i)$, which is a subset of its predecessors. The Jacobian sparsity of the flow is therefore the DAG itself.

The latents $u_1, \dots, u_p$ are standard logistic. That choice is what makes the fitted parameters interpretable, because a shift on the latent scale is then a log-odds ratio.

The components

The transformation decomposes additively on the latent scale, which is what keeps the interpretation valid. Each node's h is

$$u_i = h(x_i \mid \text{pa}(x_i)) = \underbrace{h_\vartheta(x_i)}_{\text{intercept}} + \underbrace{\sum_j \beta_{ij} x_j}_{\text{linear shifts}} + \underbrace{\sum_k g_{ik}(x_k)}_{\text{complex shifts}} + \underbrace{(\beta_0 + b_\Theta(x_{\text{mod}})) x_t}_{\text{varying coefficient}},$$

and every causal parent enters through exactly one term. Take x_5 with parents $\text{pa}(x_5) = \{x_1, x_2, x_4\}$ as the example.

- **Simple intercept, SI.** $h_\vartheta(x_5)$ has constant parameters. It is a flexible monotone baseline, a Bernstein polynomial by default, and it is the same for every observation.

- **Complex intercept, CI.** The parameters ϑ are themselves a function of some parents. The whole transformation bends with the parent, which permits interactions that an additive shift cannot express.
- **Linear shift, LS.** $\beta_{51}x_1 + \beta_{52}x_2$. One interpretable number per parent.
- **Complex shift, CS.** $g(x_4)$, an unrestricted network of the parent. It stays additive on the latent scale.
- **Varying coefficient, VC.** $(\beta_0 + b_\Theta(x_{\text{mod}}))x_t$: a treatment x_t whose effect β varies with modifier covariates through a small penalized network. `varying-coefficients.md` is its guide.

Sampling has to invert only the intercept, because the shifts move to the other side:

$$x_5 = h^{-1}(u_5 \mid x_1, x_2, x_4) = h_\vartheta^{-1}\left(u_5 - \underbrace{\beta_{51}x_1 + \beta_{52}x_2}_{\text{LS}} - \underbrace{g(x_4)}_{\text{CS}}\right).$$

The components in code

Each node declares its transformation as an additive formula of terms. The formula is the node's first argument, a + sum of terms. Each constructor names the parents its term depends on. The paper's symbols I, LS, CS, VC are the classes Intercept, LinearShift, ComplexShift, VaryingCoefficient.

formula for a continuous node x_3	$u_3 = h(x_3 \mid \text{pa})$
nothing, or <code>I()</code>	$h_\vartheta(x_3)$, the simple intercept
<code>LS("X1")</code>	$h_\vartheta(x_3) + \beta x_1$
<code>I("X1")</code>	$h_{\vartheta(x_1)}(x_3)$, the complex intercept
<code>CS("X1")</code>	$h_\vartheta(x_3) + g_1(x_1)$
<code>LS("X1") + CS("X2")</code>	$h_\vartheta(x_3) + \beta x_1 + g_2(x_2)$
<code>CS("X1", "X2")</code>	$h_\vartheta(x_3) + g_{12}(x_1, x_2)$, one joint network
<code>CS("X1") + CS("X2")</code>	$h_\vartheta(x_3) + g_1(x_1) + g_2(x_2)$, two additive networks
<code>I("X1", "X2")</code>	$h_{\vartheta(x_1, x_2)}(x_3)$, one joint network
<code>I("X1", "X2", allow_interaction=False)</code>	$h_{\vartheta(x_1) + \vartheta(x_2)}(x_3)$, one network per parent, summed in coefficient space
<code>CS("X1") + VC("X2", t="T")</code>	$h_\vartheta(x_3) + g_1(x_1) + \beta(x_2)x_T$

Three rules follow from the model.

- **Exactly one intercept, first.** A formula written without one gets `I()` prepended. `I()` without parents is the simple intercept, `I(...)` with parents the complex one; `SI()` and `CI(...)` spell that out with the arity checked.
- **Joint versus additive is argument grouping.** Several parents in one term form one network over all of them, an interaction; the same parents in separate terms act additively. For the intercept the grouping is said with `allow_interaction`, because a node takes at most one intercept with parents.
- **Every parent enters through exactly one edge-owning term.** The VC modifiers are the one exception: `CS("X2") + VC("X2", t="T")` is the intended pattern, where x_2 acts prognostically through the shift and modifies the treatment effect.

The three knobs on a term

transform= on I picks the class of h_ϑ for a continuous node:

- "bernstein", the default: a Bernstein polynomial with `n_coeffs=20` unconstrained coefficients whose tails extrapolate along the boundary slope;

- "spline": a monotone rational-quadratic spline with bins=8, whose tail slope is fixed (zuko-upstream.md explains the consequence);
- "affine": location and scale only, so the node-conditional is a logistic GLM.

Extra keyword arguments of `I` pass straight to the transform class, `I(transform="spline", bins=16)`. Each transform pre-scales the data from the training `range_q` and `$1 - $range_q` quantiles onto a fixed domain; `range_q` is an intercept option, default 0.05, and `range_q=0` uses the minimum and maximum. Ordinal nodes have no transform to pick.

input_transform= on **I**, **CS**, **VC** transforms that term's continuous network inputs, with statistics frozen at the first fit: "minmax", "standardize", or a callable `fn(x, train)`. **LS** and the **VC** treatment stay raw, so their coefficients keep their units.

units=, **activation=**, **batch_norm=** on **I**, **CS**, **VC** size the term's network. `code-map.md` lists every default.

Ordinal nodes

An ordinal node has no monotone transform to choose. Its intercept is the cutpoint vector of an ordered logit,

$$P(Y \leq k \mid \text{pa}) = \sigma(\vartheta_k - s(\text{pa})),$$

with increasing cutpoints ϑ and s the total shift. The shift is subtracted where a continuous node adds it; `notation.md` states the convention and `interpretation.md` works through what it does to the sign of a coefficient.

The log-probability of an observed level is the difference of two sigmoids, computed in log space with `logsigmoid` and a stable `log(1 - exp(x))` and taking per element the better-conditioned side. The direct difference of two sigmoids has exactly zero gradient once they saturate in float32, and a node that starts there never recovers.

Parents enter every network as features: a continuous parent raw, in one column; an ordinal parent one-hot, in one column per level. Abduction is exact for continuous nodes and truncated-logistic for ordinal ones, so `flow.sample(u=flow.abduct(df))` reproduces `df` exactly, level-exactly for ordinal nodes.

What the model cannot do

- The latent distribution is fixed to standard logistic. It is not learned.
- A **VC** treatment is continuous or a binary ordinal node; a multi-level ordinal treatment is not supported. Propensity centering needs a binary ordinal treatment.
- The DAG is an input. `tramdag` fits the mechanisms of a graph you supply, and does not discover the graph.
- Identification is the usual causal one. No hidden confounding, and the graph has to be right.

Notation

This page defines the notation. Every guide, docstring and notebook in this repository follows it.

Symbols

Random variables are capital Latin letters, for example Y . Their realizations — observed or sampled values — are lowercase Latin letters, for example y . The cumulative distribution function of Y is F_Y and the density is f_Y . A hat marks an estimate: $\hat{F}$ is the fitted distribution, F the true and often unknown one.

Sets and vectors are bold, for example $\mathbf{X} = [X_1, \dots, X_J]$.

A single optimized parameter is a lowercase Greek letter, for example ϑ or β . A vector of parameters is bold lowercase, for example $\vartheta = (\vartheta_1, \dots, \vartheta_M)^\top$. A tuple that collects several parameter groups — for example all weight matrices of a network — is bold capital, for example Θ .

A function that a parameter vector defines carries it as a subscript: h_{ϑ} is the monotone transformation with coefficients ϑ .

The model's symbols

Symbol	Meaning	In code
X_j, Y	the variables of the DAG	node names in the spec
$\text{pa}(X_j)$	the causal parents of X_j	node_parents
U_j	the standard-logistic latent of node j	<code>u = flow.abduct(df)</code>
h_{ϑ}	the monotone transformation of a node	the I term; theta in docstrings
ϑ_k	the ordinal cutpoints, elements of ϑ	ordinal_cutpoints
β	a linear-shift coefficient; e^β is an odds ratio	LS; <code>flow.ls_coefficients()</code>
g_j	a complex shift, an NN of one term's parents	CS
β_0	the constant treatment effect of a VC term	beta0
b_{Θ}	the penalized effect-modification network	the VC net; <code>units=</code> sets its layers
λ	the L2 weight on b_{Θ}	<code>penalty=</code>
σ	the logistic function	<code>torch.sigmoid</code>

Every node's model is one additive formula on the latent scale:

$$u = h(x \mid \text{pa}(x)) = h_{\vartheta(\cdot)}(x) + \sum_j \beta_j x_j + \sum_k g_k(x_k) + (\beta_0 + b_{\Theta}(x_{\text{mod}})) x_t.$$

For a continuous node the shifts are added, as written. For an ordinal node the shift is subtracted inside the sigmoid: $P(Y \leq k \mid \text{pa}) = \sigma(\vartheta_k - \text{shift})$.

Reading a fitted TRAM-DAG

A TRAM-DAG is a normalizing flow, but it is built so that parts of it stay readable. This page collects the read-outs and says what each number means. For the model itself see `model.md`, and for the symbols see `notation.md`.

Every claim on this page is checked somewhere that runs. The links say where, so nothing here is a number typed into prose.

Which term owns which edge

`flow.to_matrix()` gives the labelled adjacency matrix of the model. Rows are parents and columns are children. A cell holds the term tag: LS, CS, CI, VC for a treatment edge, or VCm for a varying-coefficient modifier. An empty cell means no edge. A multi-parent term carries its parent group as a suffix, and several terms in one cell join with +.

This is the meta-adjacency view of the paper. It is also the fastest way to confirm that the model you wrote is the model you meant. `plot_dag(spec)` draws the same thing, with one edge style per term.

Exercised in the varying-coefficients notebook.

A linear shift is a log-odds ratio

`flow.ls_coefficients()` gives `{node: {parent: weights}}` for the LS terms only. A CS or VC shift is a network and has no single weight, so those are skipped.

Two things decide how to read a weight.

The sign follows the latent-scale convention of notation.md. A continuous node adds its shift, so a parent that raises the child gets a *negative* weight. An ordinal node subtracts its shift, so the sign is the one you expect there.

The scale is the latent scale, not the data scale. For a continuous node the weights carry the transform's scaling. A single weight is therefore not comparable to a regression coefficient. Their *ratio* is comparable, because the scaling cancels. An ordinal node has no such scaling, so $\exp(\beta)$ is an odds ratio directly.

An ordinal parent gives one weight per level, from its one-hot encoding. Those weights are identified only up to a common constant. The one-hot columns sum to one in every row. Adding a constant to all of them, and subtracting it from the intercept, leaves the likelihood unchanged. Read them as differences. $w[k] - w[0]$ is the level-k-against-level-0 log-odds ratio, and it is the column that `flow.design_matrix(df, node, drop_first=True)` drops.

Checked in `notebooks/classical_fit_tram_dag.py`, which compares the weights against `statsmodels` and R, and pinned in continuous integration by `experiments/triangle_mixed.py`, whose odds-ratio check is in `paper-replication.md`.

Ordinal nodes: cutpoints and the interventional PMF

An ordinal node's intercept is its cutpoint vector, not a transform (`model.md`). `tramdag.transforms.ordinal_cutpoints` turns the unconstrained parameters into the increasing cutpoints, which is how you read a fitted baseline.

`flow.pmf(df, node, do=...)` gives the class probabilities per row, in closed form. It needs no sampling, and `do=` makes it interventional by overriding columns before the parents are read. That combination is what lets a treatment-effect question be answered exactly rather than by Monte Carlo.

Exercised in the classical-fitting notebook, and under `do=` in `experiments/triangle_mixed.py`, which is checked against committed ground truth on every run.

Continuous nodes: density and one fitted shift

`flow.density(df, node, grid, do=...)` is the continuous counterpart of `pmf`. For every row it evaluates $p(\text{node} = g \mid \text{pa})$ at each grid value, in closed form from the transform. Use it to look at a fitted conditional without sampling it.

`flow.shift_curve(node, parent, grid)` evaluates one fitted shift term along a grid of parent values. Use it to plot a CS term against the function it was supposed to learn. It goes through the term's own evaluation, so a custom term works too.

A caution on both: a CS term is identified only up to a constant, because that constant can move into the intercept. Compare a fitted curve to the truth after mean-centring both, not point by point.

The price of a complex intercept

`I(...)` with parents is the flexible end of the family. The parents control the transform parameters, so the node can bend in ways no additive shift can express. The cost is that there is no single coefficient to read.

`paper-replication.md` measures that trade-off on the paper's triangle process, fitted once with an LS edge and once with a CS edge.

A middle option exists. `I("a", "b", allow_interaction=False)` builds one network per parent and sums their parameter vectors, so the parents act additively on the transform. `flow.intercept_contributions(df, node)` then decomposes that sum into mean-centred per-term parts, using the usual additive model convention, so the parts become comparable. This is a post-hoc read-out and changes nothing about the fitted model.

Worked through in the intercept notebook.

An effect that varies with a covariate

`VC(*modifiers, t=...)` fits $\beta(\text{modifiers}) \cdot x_t$, and `flow.varying_coef(df, node)` reads the fitted β back in closed form.

`flow.scores(df, node)` and `flow.effect_modifier_scan(df, node, t=...)` come before that decision rather than after it. They rank candidate modifiers by how much the treatment coefficient's per-observation scores drift when the rows are ordered by each candidate. A cheap all-LS fit is enough, so the shortlist is measured rather than guessed.

See `varying-coefficients.md` and `scores.md`, and `notebooks/varying_coefficients.py`.

Fitting a TRAM-DAG

This page is the reference for training a `CausalFlowDAG`: the likelihood, the two fitting paths `fit` and `fit_classical`, and the hooks of the Adam path. Every recipe named here runs end to end in `notebooks/training_strategies.py`, which is the one place for the code and closes with a runtime comparison of the recipes on one workload.

One module, one sub-model per node

A `CausalFlowDAG` is a single `torch.nn.Module` holding one independent sub-model per variable: an intercept that produces the transform parameters θ , the monotone transform h with range buffers and no weights of its own, and one shift module per shift term. The nodes share no parameters. The DAG lives entirely in which parents each node reads. There is no edge weight matrix and no shared trunk. `code-map.md` says which class implements which term.

The likelihood

The flow maps iid standard-logistic latents U to the observed X in causal order, and node i reads its parents as data. The Jacobian is therefore triangular, and the joint log-likelihood decomposes per node, $\log p(x) = \sum_i \log p(x_i \mid \text{pa}(x_i))$. `CausalFlowDAG.node_log_prob` computes one term and `log_prob` sums them.

- **Continuous node.** Change of variables through the monotone transform: $u = h(x; \theta) + s$ and $\log p(x \mid \text{pa}) = \log f_{\text{logistic}}(u) + \log |h'(x)|$. The transform returns the log-derivative alongside u .
- **Ordinal node.** The ordered-logit head of `model.md`, evaluated as the log of the cutpoint-interval probability in log space.

The training loss is the summed per-node mean negative log-likelihood over the batch, plus the VC penalty. `log_prob` returns the per-row joint instead.

Because parents enter as data, the per-node gradients are independent, so a joint fit of the summed loss equals a separate fit of each node. That is what licenses per-node learning rates, freezing through a callback, and the all-LS classical fit.

Path A: stochastic optimization with fit

`fit` is the general-purpose trainer, and the only one for a model with a CS, CI or VC term. It is one minibatch Adam loop that keeps the final weights.

- **Optimizer.** One optimizer over all parameters, `Adam(lr=learning_rate)` by default; `optimizer=` takes any `torch.optim.Optimizer`, and `per_node_adam` builds per-node parameter groups.
- **Minibatches.** A fresh `torch.randperm` shuffle each epoch, seeded by `seed=`, which seeds the shuffle only and not the weight init.
- **Epochs.** `epochs` has no default: a fixed budget under-spends on one workload and wastes on the next, so every caller states its own.
- **Calibration.** The first `fit` calls `calibrate(train_df)`. Every term freezes its data-dependent state there, the intercept its `range_q` quantiles and each `input_transform=` its statistics. A loaded checkpoint is never recalibrated.
- **Marginal start.** Off by default; every simple intercept starts at zuko's zero. `fit(marginal_init=True)` and `calibrate(train_df, marginal_init=True)` set it instead: every Bernstein or ordinal simple intercept starts at the empirical marginal of its column, as `logit(F_hat)` in control points or cutpoints. The spline, the affine and the `range_q=0` transforms have no such start. It is a pure initialization and leaves the MLE unchanged.

`fit_classical` takes no such flag. The likelihood of an all-LS model has one optimum, so a start would change how far the line search travels and nothing else.

The flag rides on calibration's guard, so it applies once, on the fit that calibrates. That is what a schedule needs: a second `fit` continues training rather than discarding the intercepts the first one trained.

- **Validation.** `validation_data=` takes a frame; `validation_split=` takes a float and uses the last fraction of `train_df` unshuffled, so shuffle the frame first if its row order means anything; only the head calibrates. With either, `fit` writes the per-node validation NLL after every epoch into `flow.history["val"]`.
- **Logging.** `flow.history["lr"]` records the optimizer's rates per epoch, one dict keyed by a group's node tag or else its index: `{0: lr}` for the default Adam, `{node: lr}` under `per_node_adam`. `verbose=N` prints every Nth epoch and the last one; the default 0 is silent.
- **Callbacks.** `callbacks=` takes one `Callback` or a list. The hooks are `on_fit_begin`, `on_epoch_end` and `on_fit_end`; a bare callable is an `on_epoch_end` hook `cb(flow, epoch, optimizer)`, and any `True` return stops the fit. `on_fit_end` runs before the VC re-centering. The shipped callbacks are `EarlyStopping`, which restores the best-validation weights and takes an optional `patience`, and `PerNodeEarlyStopping` with `per_node_adam`, the same per node: a node freezes after `patience` flat epochs, loads its best weights back and optionally decays its rate on the way. `fit` refuses two callbacks that restore weights. All of them read `history["val"]`.
- **Centered VC propensities** ride the training frame as the column that `VC(center=)` names, and split and minibatch with it. `varying-coefficients.md` is the guide.

Which recipe

All-LS models train to the MLE and keep the final weights. Flexible models with a CI, CS or VC term overfit observational confounding at the MLE and need the best-validation weights to recover the causal effect.

recipe	when
<code>fit_classical</code> plain Adam	all-LS spec: exact, deterministic, seconds an all-LS spec that <code>fit_classical</code> refuses, quick looks
multi-phase Adam, one fit call per rate <code>EarlyStopping()</code>	a tighter MLE without a scheduler any CI, CS or VC model; register it, <code>fit</code> has no default
<code>EarlyStopping(patience=)</code> a global plateau Callback around torch's <code>ReduceLROnPlateau</code>	also stop once the best epoch is that old one shared decaying rate
<code>PerNodeEarlyStopping(patience=,</code> <code>lr_patience=)</code> with <code>per_node_adam</code>	nodes converge at different speeds; self-stopping

Two details are easy to get wrong.

- A second fit call continues training and history accumulates. That is what makes a multi-phase schedule a loop of fit calls.
- A post-fit `load_state_dict` skips the VC re-centering. Restore weights from `on_fit_end` instead, as `EarlyStopping` does.

Path B: classical optimization with `fit_classical`

`fit_classical` is the optimizer for all-LS models, where every node-conditional is an ordered logit or a Colr model. It raises on any CS, CI or VC term.

- **Full-batch, float64, L-BFGS** with a strong-Wolfe line search; no minibatches, schedule or early stopping, so the same init gives bit-identical results. A converged fit matches statsmodels and R to about four decimals, where a converged Adam fit gets to about $1e-3$.
- **Budget.** `max_iter=400` is a default, not a promise. Torch ends the run when the NLL or the parameters move by less than $1e-9$. A model with several nodes can need thousands of iterations to stop on that rule. Give the budget room and read the report.
- **The report.** `stop_reason` is "tolerance", "max_iter" or "max_eval" (torch's budget of closure calls, `max_iter * 5 // 4`). converged needs both: the run stopped on its own AND `grad_norm` is at most `tramdag.fitting.GRAD_TOL` ($1e-2$). Both conditions are necessary, because the same tolerance fires when the line search stalls far from the optimum. Such a run stops on "tolerance" with a large gradient norm and is not converged.
- **float64 is transient.** The fit restores float32 afterwards, and checkpoints stay float32.
- **Weakly identified directions never settle.** A Bernstein intercept, rare one-hot levels or a flat treatment-effect ridge drift along zero-curvature valleys after the likelihood is at its optimum, which is why `tolerance_grad` is 0 and the gradient bound is loose. Correctness comes from the comparison with classical software in `notebooks/classical_fit_tramdag.py`.

A converged `fit_classical` leaves the model at the MLE, ready for any operation. A fit call from there stays put, which is both a check that the classical solution is the optimum and a warm start; the VC guide uses it for `beta0`.

Memory and disk

Neither path writes to disk. The parameters, the optimizer state and the history dict live in RAM, and whatever a callback records is yours. The only disk I/O is the explicit `save()` and `load()`.

Per-observation scores and the effect-modifier scan

`flow.scores(df, node)` returns the score $\psi_i = \partial \ell_i / \partial \theta$ of each observation for the node’s interpretable shift coefficients: every LS weight and every VC term’s β_0 . An LS weight gives one column per continuous parent and one per one-hot level of an ordinal parent; the β_0 column is named after the treatment.

The computation is analytic, with no autograd. Shifts enter the latent additively, so $\partial \ell_i / \partial \beta = (\partial \ell_i / \partial s_i) x_i$ with $\partial \ell_i / \partial s$ in closed form. The function is a pure read-out that touches neither fitting nor sampling. At a fitted MLE each column sums to about zero, which the tests pin together with a float64 finite-difference check.

The effect-modifier scan

`flow.effect_modifier_scan(df, node, t=)` is the cheapest effect-modifier detector available. It applies the structural-change logic of model-based recursive partitioning (Zeileis & Hornik; Dandl et al. 2024) to the treatment-coefficient scores of a cheap all-LS fit, which `fit_classical` delivers in seconds.

For each candidate covariate the scan orders the scores by that covariate and forms the scaled cumulative sum $B_j = \sum_{i \leq j} \psi_{(i)} / (\text{sd}(\psi) \sqrt{n})$. Under a stable coefficient B is a Brownian bridge, so $\sup |B|$ has the Kolmogorov distribution, with the 5 % critical value 1.358. A coefficient that varies with the covariate makes the ordered scores drift. The result is one row per candidate with the statistic, its p-value, the critical value and a flag; the flagged covariates are the shortlist of VC modifiers to measure (varying-coefficients.md).

Read it as screening, not as a test of effect modification. The statistic measures how unstable the cheap model is along a covariate, and instability has two sources: a coefficient that genuinely varies, and a prognostic part the cheap model gets wrong. A prognostic-only covariate with a linear effect stays unflagged; give it a quadratic effect and it flags as strongly as the true modifiers. A flag therefore means “look here”, and what you find is a modifier or a misspecification, both worth knowing. `notebooks/varying-coefficients.py` shows both cases side by side.

Details: for a binary ordinal LS treatment, `t` resolves to the identified level-1-against-0 contrast, and for a VC treatment to β_0 . Candidates default to every column of `df` except the node and `t`, and need not be parents. For few-level, heavily tied candidates the ordering is only partial, so read the scan as a ranking diagnostic rather than an exact-size test. The scores also serve influence-function analyses and robust standard errors.

Varying-coefficient treatment effects: `VC(*modifiers, t=, penalty=, center=)`

A VC term gives a node a treatment-effect head with its own bias-variance budget. It contributes $\beta(x) x_t$ with $\beta(x) = \beta_0 + b_\Theta(x)$ to the node’s shift, where b_Θ is a deliberately small, penalized network of the modifiers. The point is not expressiveness. The point is that the flow estimates the effect function with care, not as a by-product. Every claim here runs in `notebooks/varying-coefficients.py`, which is the one place for the code.

Modifiers may appear twice in a node, as prognostic parents through CS or LS and as effect modifiers through VC. That pattern is intended. Only the treatment named by `t=` owns its edge, and a second term that declares it raises an error.

Why not a joint CS over treatment and modifiers

For a binary treatment the multi-parent CS is equivalent in expressiveness, since any shift decomposes as $s(x, t) = s(x, 0) + [s(x, 1) - s(x, 0)] t$. But the CS form has no effect-specific regularization.

The likelihood rewards a good average fit of $s(x, t)$, and nothing rewards a smooth difference between the arms, so the read-out is the difference of two jointly fitted, unregularized networks and amplifies noise. On the `vc_hetero` process the CS reduced form reaches a correlation of about 0.5 with the true effect function even though the model is in-class; the VC term reaches about 0.99 on the same protocol. Causal forests and R-learners work not because they target the effect but because they regularize it (Nie & Wager 2021; Athey, Tibshirani & Wager 2019). VC brings that ingredient into the TRAM framework.

Semantics

- **Scale.** $\beta(x)$ lives on the node's latent scale, added for a continuous node and subtracted from the cutpoints for an ordinal one, exactly like an LS weight. With no modifiers, `VC(t="T")` is `LS("T")` bit-exactly, so VC against LS is a nested question.
- **Penalty.** The objective is $\sum_i \text{NLL}_i + \lambda \|b_\Theta\|^2$ on the total-NLL scale, with `penalty=` as λ . It is a fixed Gaussian prior whose shrinkage vanishes as n grows, it never applies to β_0 , and $\lambda \rightarrow \infty$ recovers the LS fit. Raise it when the modifiers are many or n is small.
- **Identification.** A constant moves freely between β_0 and b_Θ . The head's output layer is zero-initialized, so $\beta(x) = \beta_0$ at step 0, and after fit the flow re-centers the head to mean zero over the training data, which preserves the function. β_0 is therefore the main effect in the training population, the Colr reading when β is constant.
- **Warm start.** Fit the all-LS version classically and copy the treatment weight into `beta0` before fit, so training starts at the classical answer and learns only deviations.
- **Treatments.** Continuous or binary ordinal, entering as its 0/1 level so that β is the identified level-1-against-0 contrast. The term is linear in the treatment.
- **Read-out.** `flow.varying_coef(df, node)` evaluates $\beta(x)$ in closed form, deterministic and free of the outcome. For a binary treatment it equals the abduction difference $u(x, 1, y) - u(x, 0, y)$ identically, which a test pins.

Propensity centering: `center="col"`

With `center=` the term contributes $\beta(x) (t - \hat{e}(x))$ instead of $\beta(x) t$. This is Robinson's R-learner orthogonalization inside the likelihood, the ingredient Dandl et al. (2024) found decisive for effect estimation under confounding in model-based forests. The finding reproduces here on the confounded process, where the model deliberately under-specifies its prognostic part: the uncentered $\hat{\beta}$ absorbs the confounded misfit, and centering brings it back near the truth. The notebook reports the measured reduction and `tests/test_vc_centered.py` requires at least a factor of two. If the prognostic part is correctly specified, centering changes little. It is insurance against the misspecification you do not know you have.

The design is two-stage and frozen, because the naive versions are wrong.

- **Training** uses out-of-fold propensities that you compute and pass as the training-frame column `center=` names, one value per row. Any propensity model works as long as each fold is predicted by a fit that never saw it, the cross-fitting requirement of double machine learning. In-sample propensities reintroduce the own-observation bias and can be worse than no centering. The values enter the loss as frozen data, so no gradient reaches the treatment node and the per-node factorization stays intact. `fit` refuses a centered spec whose frame lacks the column.
- **Inference**, in `log_prob`, `sample`, `abduct`, `pmf` and `scores`, recomputes $\hat{e}$ from the flow's own fitted treatment node on the current parent values, detached. Under `do(T=t)` the regressor becomes $t - \hat{e}(x)$. Nothing is cached.
- **Interpretation.** With centering, β_0 is the effect at the treatment margin. `varying_coef` is unchanged, because centering moves the regressor and not β . Centering requires a binary ordinal treatment.

Validation

The `vc_hetero` process in `tests/confstest.py` is a logistic-shift SCM with a known $\beta(x) = -1 + 0.8 X_2 - 0.6 X_3$, a nonlinear prognostic part and confounded assignment, where X_2 is confounder and modifier at once. That is the configuration in which the CS reduced form fails hardest. `tests/test_vc_term.py` requires a recovery correlation of at least 0.9 at $n = 5000$, a fitted β_0 that matches `fit_classical` under a large penalty, and the read-out identities. The centering claims are measured on the confounded process in the same file. `scores.md` covers the scan that shortlists modifiers before a VC term is declared.

Paper replication: protocol, hyperparameters and results, per experiment

The eight variants under `experiments/` replicate the TRAM-DAG paper (Sick & Dürr, CLeaR 2025, arXiv:2503.16206) against its own R code (`tensorchiefs/tram-dag`). For each experiment this page lists the DGP, the model, every hyperparameter with its source, what deviates from the paper and why, and the numbers. The paper states four training numbers: $n = 40000$, 500 epochs, Adam, and Bernstein order 20; it shows its results as figures, so where it gives no number the “paper” column names the figure and what it shows. The pinned ground truth is `experiments/ground_truth/*.json`; `experiments/README.md` explains the YAML variants, the frozen data and the check.

Seeds are DGP 42, init 7 and shuffle 0. The triangle scripts of the reference run unseeded and the comparison scripts seed R’s RNG, which torch cannot replay, so every seed here is a repository choice.

What is common to all eight

item	reference	here
latent	standard logistic, shifts added on the continuous scale, subtracted for ordinal $P(Y \leq k) = \sigma(\vartheta_k - s)$	same (<code>notation.md</code>)
Bernstein basis	<code>len_theta</code> unconstrained coefficients, <code>to_theta</code> softplus-cumsum, domain $\rightarrow [0, 1]$ with tangent-linear extrapolation outside; the domain comes from the train 5 %/95 % quantiles in the triangle scripts (<code>quantile(..., c(0.05, 0.95))</code>) and from min/max in the comparison scripts (<code>scale_df</code>)	zuko Bernstein, <code>n_coeffs</code> unconstrained (zuko ties two control points on, so <code>n_coeffs=20</code> is order 21 against the reference’s order 19; the free-parameter count is what matches), domain = <code>train range_q/1-range_q</code> quantiles $\rightarrow [-5, 5]$, linear extrapolation. Triangle: <code>range_q: 0.05</code> , a match up to the reparametrization. CAREFL: <code>range_q: 0</code> , the reference’s min/max. VACA: quantiles kept, deviation D1, measured below

item	reference	here
networks	triangle scripts create_param_net with hidden_features = c(2, 25, 25, 2) continuous and c(2, 2, 2, 2) mixed, sigmoid (the ReLU line is commented out); the vector reads as in/out dims around the hidden stack, so the hidden layers are (25, 25) and (2, 2). Comparison scripts make_model: dense(10, tanh) → dense(100, tanh) → dense(len_theta), one net per node	units and activation set per variant to exactly those stacks. The package defaults in code-map.md replicate the PyTorch reference buehlpa/TramDag instead and are not the paper's nets
init	triangle scripts: LinearMasked layers with Keras random_normal (N(0, 0.05 ²)) on weights and biases, the LS beta layer included; comparison scripts: layer_dense default, glorot-uniform weights and zero biases	init: normal (triangle) and init: glorot (VACA/CAREFL) through CausalFlowDAG(init=); torch's default init remains the framework default, and under the full-batch protocol the init decides the fit (see VACA)
optimizer	Keras Adam, eps 1e-7	torch Adam, eps 1e-8; measured: no effect (VACA identical to four digits)
calibrated start	none	marginal_init: true for the two triangle LS models, atan-cs and exp-cs, deviation D4 below; off where it moves the endpoint (linear-cs, sin-cs, VACA) and impossible for CAREFL's range_q: 0 domain. calibrate without the flag never touches the weights
intercept output layer	Keras dense with bias	bias-free, deviation D3: the same function class, because the bias adds a constant to all unconstrained coefficients
plateau rule (VACA/CAREFL)	update_learning_rate: one optimizer, reduce when the summed validation NLL has not improved for 50 epochs (strict <), factor 0.1, min 1e-7	torch ReduceLR0n- Plateau(patience=49, threshold=0, threshold_mode="abs", factor=0.1, min_lr=1e-7) on the summed history["val"], the same rule, verified against torch's source; experi- ments/helpers.py::fit_paper drives it

Triangle, continuous (triangle.py): paper Sec. 6.1, App. C.3

DGP (summerof24/triangle_structured_continuous.R):

- $x_1 \sim 0.5 N(0.25, 0.1^2) + 0.5 N(0.73, 0.05^2)$
- $h(x_2 | x_1) = 5 x_2 + 2 x_1 = u_2$
- $h(x_3 | x_1, x_2) = 0.63 x_3 - 0.2 x_1 - f(x_2) = u_3$, $u \sim \text{logistic}$

f has three forms: linear $-0.3 x$, atan $0.75 \text{atan}(5 (x + 0.12))$, $\sin 2 \sin(3x) + x$. True weights in the flow's convention: $\beta_{12} = +2$, $\beta_{13} = -0.2$, $\beta_{23} = +0.3$ (linear only). A fitted CS learns $-f(x_2) + \text{const}$.

Model: x_1 : SI, x_2 : SI + LS(x_1), x_3 : SI + LS(x_1) + {LS(x_2) | CS(x_2)}, Bernstein, the CS net hidden (25, 25) with sigmoid.

hyperparameter	paper / R code	here
train / validation	dgp(40000) / dgp(40000), two draws	40000 / 40000, two draws
epochs	500	300 (linear-cs: 500), one continuous run; the CI deviation
lr	0.001 (optimizer_adam() default)	0.004, the CI deviation
batch	32 (Keras fit() default)	256, the CI deviation
len_theta / n_coeffs	20	20
schedule / early stop / init	none / none, final weights / random_normal	none / none / init: normal
coefficient read-out	after every epoch (Keras loop)	fit(callbacks=), every epoch

Results

variant	metric	paper	paper protocol, init: normal (batch 32 / lr 0.001 / 500 epochs)	CI config, the pinned ground truth
linear-ls	$\beta_{12} / \beta_{13} / \beta_{23}$	Fig. 14 trajectories; App. C.3 text: 1.98 / -0.21 / 0.26	1.987 / -0.170 / 0.282	1.981 / -0.161 / 0.281
linear-cs	β_{13} ; max $ \hat{g} - (-f) $ on $[-1, 1]$	Fig. 17: fitted CS is a straight line	-0.173; 0.122	-0.178; 0.088
atan-cs	β_{13} ; cs max err	Fig. 7 right / 15 / 16: CS on $-f$; text: $\beta_{12} = 2.07$, $\beta_{13} = -0.203$	-0.168; 0.059	-0.168; 0.108
sin-cs	β_{13} ; cs max err	Fig. 18: CS follows the non-monotone f on $x_2 \in [-1, 0]$	-0.195; 1.10	-0.168; 0.240 on the paper's $[-1, 0]$ window
all	$ E[x_3 \text{do}(x_1 = -1)] \text{ flow} - \text{DGP} $	Figs. 16/17: histograms overlap	0.09-0.14	0.08-0.11
all	val NLL x_3	—	2.4607-2.4764	2.4606-2.4764

β_{13} measures -0.16 to -0.18 instead of -0.2 . β_{13} multiplies x_1 , whose two mixture components sit at 0.25 and 0.73 , so x_1 has sd 0.254 against 0.375 for x_2 and 2.918 for x_3 , and $SE(\beta_{13}) \approx 0.036$ at $n = 40000$. The value stays within about one SE, and β_{13} is too weakly identified at $n = 5000$ for the frozen 5000-row CSVs to serve as fit checks; the fit checks train on the paper’s $n = 40000$ protocol.

The (25, 25) hidden stack is what lands sin-cs on Fig. 18, including its small -1 -endpoint deviation; a literal [2, 25, 25, 2] reading puts a 2-sigmoid bottleneck on the input and cannot reproduce the figure at any protocol (cs max err 1.22).

Triangle, mixed (triangle_mixed.py): paper Sec. 6.2, App. C.4, App. B

DGP (triangle_structured_mixed.R): x_1 and x_2 as above. x_3 is ordinal with four levels and cutpoints $\theta = (-2, 0.42, 1.02)$ from the R code, which the paper does not state. The level is $\#\{k : u_3 > \theta_k + 0.2 x_1 + f(x_2)\}$, with f either linear $-0.3 x$ or exp $0.5 \exp(x)$. The paper adds the ordinal shift and the flow subtracts it, so the fitted weights are $\beta_{13} = -0.2$, $\beta_{23} = +0.3$.

Model: x_1 : SI, x_2 : SI + LS(x_1), x_3 : OrdinalNode(4, LS(x_1) + {LS(x_2) | CS(x_2)}), the CS net hidden (2, 2) with sigmoid.

hyperparameter	paper / R code	here
train / validation epochs, lr, batch, schedule, init	dgp(40000) / dgp(10000) 500, 0.001, 32, none, random_normal	40000 / 10000, two draws 350 (exp-cs) / 200 (linear-ls) continuous, none, init: normal; lr 0.004 (linear-ls: 0.002) / batch 256, the CI deviations
n_coeffs (x1, x2)	20	20
odds-ratio check (App. C.4)	odds($x_2 \leq -1$) under do($x_1 += 1$), theory $e^2 \approx 7.39$	40000 rows, seed 99
counterfactual PMF (App. B)	Fig. 10 explains why point counterfactuals fail for the ordinal node	2000 rows $\times$ 200 draws

Results

variant	metric	paper	paper protocol, init: normal (500 epochs)	CI config, pinned
linear-ls	$\beta_{12} / \beta_{13} / \beta_{23}$	Fig. 19: 2 / -0.2 / 0.3	1.987 / -0.246 / 0.319	1.978 / -0.258 / 0.333
linear-ls	odds ratio predicted / DGP	$e^2 \approx 7.39$; C.4 text: 7.74, CI [7.16, 8.38]	7.29 / 7.19	7.23 / 7.19
linear-ls	CF PMF TV vs analytic; P(true level) flow / analytic / mode bound	— (App. B qualitative)	0.044; 0.718 / 0.728 / 0.806	0.050; 0.716 / 0.728 / 0.806
exp-cs	β_{13} ; cs max err	Fig. 20: distributions match	-0.207 ; 0.142	-0.200 ; 0.156
exp-cs	CF PMF TV; P(true level)	—	0.023; 0.917 / 0.921	0.024; 0.920 / 0.921

VACA / CNF benchmark (vaca.py): paper Sec. 5.1-5.2, App. C.1

DGP (Sanchez-Martin et al. 2022, App. E.1): $x_1 \sim 0.5 N(-2, 1.5) + 0.5 N(1.5, 1)$, $x_2 = -x_1 + N(0, 1)$, $x_3 = x_1 + 0.25 x_2 + N(0, 1)$. The noise is Gaussian, outside the logistic-latent family, and the all-CI flow must fit it. The analytic target is $E[x_3 \mid \text{do}(x_2 = a)] = -0.25 + 0.25 a$. The paper’s Sec. 5.2 text says $a \in \{-3, -2, 0\}$; its Fig. 5 panels and the R code (vaca_triangle.r) intervene at $a \in \{-3, -1, 0\}$, and this repository follows the code. $\text{do}(x_2 = -3)$ is off-manifold extrapolation and takes a looser tolerance.

Model: x_1 : SI, x_2 : CI(x_1), x_3 : CI(x_1, x_2), Bernstein $n_coeffs = 31$ (reference $M = 30$, $len_theta = 31$), the make_model nets.

hyperparameter	R code	here
train / validation epochs	nTrain 2500 / dgp(5000) 10000 (Figure_Triangle_Linear_Bimodal.R, the sourcing script, not in our copy of the R code, so EPOCHS/M/nTrain rest on that reading)	2500 / 5000, two draws 10000, one run, 1:1
lr	0.001	0.001
batch	full batch (one apply_gradients per epoch)	2500 = n_train
schedule	the plateau rule above	the same rule; it fires at about epoch 9050 and freezes an all-bounds point
input scaling	scale_df: everything min-max to [0, 1]	input_transform: minmax on the CI terms
Bernstein domain	train min/max (scale_df)	5 %/95 % quantiles (range_q: 0.05), deviation D1: at seed 7 the reference’s min/max domain scores 0.289 / 0.040 / 0.067 against the quantiles’ 0.096 / 0.080 / 0.022, worse at $\text{do}(x_2 = -3)$ and $\text{do}(x_2 = 0)$. CAREFL, same nets, measures the opposite way
n_compare	—	50000

Results: the check is the flow’s error against the analytic mean, not a pinned flow value.

metric	paper	here, the pinned ground truth
$ E[x_3 \mid \text{do}(x_2 = -3)] - (-1.0) $	Fig. 5: densities overlap	0.096
$ E[x_3 \mid \text{do}(x_2 = -1)] - (-0.5) $	Fig. 5	0.080
$ E[x_3 \mid \text{do}(x_2 = 0)] - (-0.25) $	Fig. 5	0.022
sd(x_1) flow vs analytic 2.0767	Fig. 4: bimodal x_1 fitted (the default CNF fails)	2.036, error 0.040
val NLL x_3	—	1.4427

The result is seed-sensitive at the off-manifold point $\text{do}(x_2 = -3)$, where the error spans 0.03–0.27 over four init draws, against 0.005–0.026 at $\text{do}(x_2 = 0)$. The paper shows one run’s densities. The committed bound is $2.5\times$ the seed-7 measurement.

The table below traces what each protocol ingredient is worth under the reference protocol, one change at a time, on the paper text’s grid $-3 / -2 / 0$ with min-max inputs unless the row says otherwise.

variant	error	reading
raw parents into the nets	0.731 / 0.426 / 0.017	the tanh nets saturate: 40 % of the rows have $ x_1 > 2$, 43 % $ x_2 > 2$
global plateau, torch init	0.523 / 0.334 / 0.129	the summed NLL keeps improving through x_1 while x_3 overfits
... without any plateau	0.562 / 0.354 / 0.142	the rule is what stops the full-batch overfit
... Adam eps 1e-7	0.523 / 0.334 / 0.129	no effect
... Bernstein on train min/max (D1 off)	0.154 / 0.012 / 0.062	helps, not the cause
... glorot init	0.035 / 0.006 / 0.007	the cause (a different random draw than the config’s seed 7)
... glorot + min/max	0.035 / 0.056 / 0.057	

A probe of the reference trajectory shows the $\text{do}(x_2)$ errors descend until epoch 6000-8500 and then creep back up; the plateau anneal fires at about 9050 and freezes them. The reference protocol’s quality is its anneal landing inside that window. Rejected alternatives, each measured: relu with raw parents wanders 0.17-0.45 on $\text{do}(x_2 = -3)$ and holds the bound only at isolated epochs; tanh on raw parents saturates outright at 0.731; minibatch stepping (batch 256, even with minmax + tanh) converges with a systematic -0.11 common-mode offset of all three do -means while the observational fit stays perfect.

CAREFL benchmark (carefl.py): paper Sec. 5.3, App. C.2

This benchmark is the reference run 1:1 on the reference’s own data. `carefl_fig5.r` sets `USE_EXTERNAL_DATA = TRUE` and trains on CAREFL’s own committed 2500 rows (`X.csv`, x_3/x_4 sd-standardized by 6.0104/1.9114) with `val = train`. CAREFL’s repository also commits the observation `x0bs.csv`, the analytic truth curves and its own predictions on the grid `seq(-3, 2.9, 0.1)`. This repository freezes all of it under `experiments/data/carefl-cf`, external input with no generator, so the Fig. 6 curves are comparable point by point and every metric is in the reference’s standardized units.

DGP (Khemakhem et al. 2021): $x_1, x_2 \sim \text{Laplace}(0, 1/\sqrt{2})$, $x_3 = x_1 + 0.5 x_2^3 + \varepsilon$, $x_4 = -x_2 + 0.5 x_1^2 + \varepsilon$ with $\varepsilon \sim \text{Laplace}(0, 1/\sqrt{2})$, x_3/x_4 divided by their sample sds. Counterfactuals are analytic by noise abduction. The observation is `x0bs.csv` = (2, 1.5, 0.8465, -0.2616); the paper prints (2, 1.5, 0.81, -0.28), slightly off it. The queries are $x_3^{\text{cf}} \mid \text{do}(x_2 = \alpha)$ and $x_4^{\text{cf}} \mid \text{do}(x_1 = \alpha)$ on the committed grid.

Model: x_1, x_2 : SI, x_3, x_4 : CI(x_1, x_2), Bernstein `n_coeffs` = 31, the same `make_model` nets as VACA, `range_q`: 0 for the reference’s `scale_df` min-max Bernstein domain.

hyperparameter	R code (carefl_fig5.r)	here
train / validation	CAREFL’s own <code>X.csv</code> , sd-standardized, <code>val = train</code>	the same committed <code>X.csv</code> , <code>val = train</code> ; a fresh standardized draw is scored, never trained or annealed on
epochs, lr	7000 @ 0.001	7000 @ 0.001

hyperparameter	R code (carefl_fig5.r)	here
batch, schedule, input scaling, init	full batch, plateau 0.1/50/1e-7, scale_df, glorot	same
Bernstein domain scoring	train min/max (scale_df) the single x_obs, curves over α (Fig. 6)	train min/max (range_q: 0) the same, plus 300 held-out rows at $\alpha \in \{-1.5, 0, 1.5\}$, all in standardized units

Results, in standardized units.

metric	paper / CAREFL	here, the pinned ground truth
Fig. 6 max $ x_3^{cf} \text{ error} $	Fig. 6: flow tracks the DGP curve; CAREFL's committed x_3 preds err up to ~ 0.7 at $\alpha = -3$	0.066
Fig. 6 max $ x_4^{cf} \text{ error} $	CAREFL's committed x_4 preds: max 0.174	0.204, at the $\alpha = -3$ grid edge; 0.074 at $\alpha = 0$
held-out CF MAE x_3 , $\alpha = -1.5$ / 0 / 1.5	—	0.015 / 0.009 / 0.013
held-out CF MAE x_4 , $\alpha = -1.5$ / 0 / 1.5	—	0.080 / 0.027 / 0.048

Three ingredients carry the agreement, each measured with everything else held fixed. Training on the committed rows instead of a fresh 2500-row draw cut the Fig. 6 x_4 max from 0.40 to 0.34 in standardized units. The reference optimization, 7000 @ 0.001 with `val = train`, fixed the parabola bottom (error at $\alpha = 0$ from 0.21 to 0.06). The min-max domain cut the grid edges (x_4 0.37 $\rightarrow$ 0.20, x_3 0.20 $\rightarrow$ 0.07). On fresh draws the parabola bottom sits below the truth near $\alpha = 0$ for a reason that is neither protocol nor init: finite-sample variance of the Laplace 12 % quantile at which the observed noise sits, in the sparse region $x_1 = 2$ with about 190 nearby rows; CAREFL's own committed draw is a mild one, which is one more reason to train on it.

Rejected alternatives, each measured: raw parents saturate the sigmoid on the Laplace parents as tanh does; relu underfits x_3 (val NLL 1.46–1.47 against the 1.403 ± 0.05 band) and grows a fragile Bernstein tail, with one held-out row inverted to $x_4 \approx 580$; minibatch underweights the sparse x_2 tail where the Fig. 6 grid ends (cf MAE x_3 at $\text{do}(x_2 = 1.5)$ 0.40 against the 0.319 bound, and the Fig. 6 x_3 error 5.9 against the full-batch 2.7).

D4: the marginal start, measured per variant

`marginal_init` sets every simple intercept to its column's empirical marginal before the first epoch (Bernstein: the control points follow $\text{logit } \hat{F}$; ordinal: the class log-odds). The reference starts `bernp$beta` at zero. Measured on this machine at the CI protocol, one run per variant with the start off and on, seeds unchanged:

variant	epochs to within 0.01 of the final train NLL, off $\rightarrow$ on	endpoint, off $\rightarrow$ on	verdict
triangle linear-ls	28 $\rightarrow$ 5	identical (β to 4 digits, val NLL 2.4606)	on

variant	epochs to within 0.01 of the final train NLL, off → on	endpoint, off → on	verdict
triangle linear-cs	28 → 5	cs max err 0.025 → 0.056; do(x1) err 0.132 → 0.080	off: the misspecified line is what this variant measures, and the start bends it on
triangle atan-cs	28 → 6	cs max err 0.108 → 0.074; do(x1) err 0.095 → 0.072	
triangle sin-cs	34 → 10	cs max err 0.240 → 0.088; do(x1) err 0.075 → 0.201, past the 0.188 bound	off: a different optimum, better curve, worse L2
mixed linear-ls	42 → 9	identical	on
mixed exp-cs	28 → 6	cs max err 0.143 → 0.060; do(x1) err 0.018 → 0.010	on
VACA	within 0.5: 2245 → 168	val NLL 1.4496 → 1.4348; do(x2 = 0) err 0.019 → 0.101, past the 0.044 bound	off: the plateau anneal fires elsewhere and freezes a different point
CAREFL	—	—	impossible: range_q: 0 has no marginal start

The start is a pure initialization, so where the optimizer reaches the same basin the endpoint is identical and only the epoch count changes. Where the model has a network shift or the anneal decides the endpoint, the basin changes, and the four such cases split two to two. The pinned ground truth of the four variants that switched on is unchanged: every metric stays within its tolerance, and the check’s advisory notes on their bounds are listed for the next deliberate re-pin.

Runtime and the CI deviations

The reference protocol’s cost is the motivation for every deviation. The triangle runs 500 epochs × 1250 steps of batch 32; one epoch takes 3.2 s single-process at 2-4 torch threads and 5.5 s at 32 threads, because the step is overhead-bound, so one variant takes 42-69 min on the 2-core CI runners. The triangle scripts therefore use batch 256 at lr 0.004, 8× fewer optimizer steps per epoch, chosen from this grid at the paper’s 500 epochs with glorot init. Every entry is the cs max err unless the column says otherwise.

batch / lr / epochs	linear-ls β13	linear-cs	atan-cs	mixed exp-cs (TV)	steps vs paper
32 / 0.001 / 500 (paper)	−0.170	0.110	0.080	0.071 (0.019)	1×
128 / 0.001 / 500	−0.170	0.170	0.041	0.126 (0.023)	1/4
128 / 0.004 / 500	−0.178	0.160	0.094	0.070 (0.014)	1/4
128 / 0.004 / 250	−0.170	0.147	0.070	0.131 (0.035)	1/8

batch / lr / epochs	linear-ls β_{13}	linear-cs	atan-cs	mixed exp-cs (TV)	steps vs paper
256 / 0.004 / 500 (CI)	-0.171	0.132	0.073	0.072 (0.017)	1/8
256 / 0.008 / 500	-0.181	0.163	0.113	0.073 (0.017)	1/8
512 / 0.010 / 500	-0.169	0.183	0.104	0.119 (0.015)	1/16

Batch 256 / lr 0.004 is the only row that keeps every cs error within about 0.02 of the paper protocol. Larger batches or rates bend the misspecified linear-cs curve (0.16–0.18), and lr 0.008 hurts atan-cs. At 500 epochs a triangle job takes 7–11 min instead of 42–69.

The epoch floors, measured at that batch and rate:

- **Triangle** holds every bound at 300 epochs; at 150 the run fails atan’s do(x1) bound (0.235 > 0.206). linear-cs keeps 500, because at 300 its fitted cs curve flattens past $|x_2| > 0.5$ (max err 0.140 against 0.088): the DGP line spans only ± 0.3 , and the sigmoid net’s slow tail convergence is half of that. relu is out, because the 2-unit input layer dies with it (atan: $\beta_{13} + 0.53$, cs err 1.42).
- **Triangle-mixed** exp-cs has two slow parts, the 2-unit sigmoid CS net and the do(x1) mean: at 100 / 250 / 350 epochs the cs max err is 0.344 / 0.177 / 0.156 and the do err 0.033 / 0.060 / 0.022, so it trains 350. linear-ls has no CS net to wait for and trains 200 epochs at lr 0.002, which reads the weakly identified β_{13}/β_{23} slightly closer to the 500-epoch values than lr 0.004 does. relu dies at the sd-0.05 normal init (cs err 0.859, $\beta_{13} - 0.055$).
- **VACA and CAREFL** run their references 1:1; the rejected shortcuts are in their sections.

Repository choices the paper does not state

- the seeds,
- the validation draw sizes for the triangle scripts (the R code’s test),
- `n_compare`,
- `n_heldout` = 300 and the α set $\{-1.5, 0, 1.5\}$, scored next to the single `x_obs`,
- the mixed cutpoints from the R code,
- the odds-ratio sample (40000 rows),
- the counterfactual-PMF sample (2000 rows $\times$ 200 draws).

Code map — every module of `src/tramdag/` — the public names and the private plumbing

This document has one entry per name, with its role and its place in the pipeline. Names in parentheses are private machinery. They are useful to know, and they are not part of the API. The last section lists every training hyperparameter and where it lives.

`spec.py` — declare the model

Every term is a `Term` subclass under its pythonic name. The paper’s symbol is the same object, so `LS` is `LinearShift`.

Name	Role
Term	One additive term of a node's transformation, plain data: a subclass per term with name and module as class attributes and its options as the keyword arguments of <code>__init__</code> assigned to <code>self</code> , so <code>options()</code> is <code>dict(vars(term))</code> . <code>+</code> on terms builds plain lists. Carries the spec-level rules check, <code>edge_parents</code> , <code>cells</code> , <code>classical</code> and <code>from_serialized</code> . The contract is in <code>architecture.md</code> .
SI()	The parentless intercept — the paper's SI. Free transform parameters, the same for every row. Carries the transform choice (<code>transform=</code> , default "bernstein"); extra keyword arguments pass straight to the transform class.
CI()	The parent-conditioned intercept — the paper's CI: the parents reshape the monotone transform. Needs at least one parent. Also carries <code>units=</code> and <code>allow_interaction=</code> (joint vs. additive multi-parent intercept).
Intercept / I	The intercept term class: without parents the paper's SI, with parents the CI; <code>SI()</code> / <code>CI()</code> are the two spellings with their arity checked.
LinearShift / LS	Linear shift βx — the interpretable log-odds coefficient. Exactly one parent.
ComplexShift / CS	Complex shift: an NN $g(x)$, additive on the latent scale. Several parents form one joint network.
VaryingCoefficient / VC	Varying-coefficient shift $(\beta_0 + b_{\Theta}(\text{mod})) x_t$ — the penalized treatment-effect head. <code>center=</code> adds propensity centering.
ContinuousNode	Continuous variable: monotone 1-D transform plus shifts. <code>terms</code> is the first positional argument.
OrdinalNode	Ordinal variable with levels classes: ordered logit (cutpoints) plus shifts.
node_parents()	Ordered de-duplicated parent names of a node (the canonical term list is <code>node.terms</code>).
validate_and_sort()	Edge-ownership validation plus Kahn topological sort. The returned order makes the flow triangular.
spec_to_dict() / spec_from_dict()	Checkpoint (de)serialization. A term serializes as <code>{term, parents, options}</code> with every option it carries; a hand-written spec may name only some, and the constructor fills the rest. A built-in term's <code>term</code> key is its name; a custom term writes its <code>module.ClassName</code> import path, which <code>_term_class</code> imports. <code>spec_from_dict</code> rejects a term without options, the node constructors normalize the formula, and <code>validate_and_sort</code> checks the DAG.
(<code>_normalize_terms</code> , <code>_check_term</code> , <code>_term_class</code> , <code>_checked_input_transform</code> , <code>_serialized</code>)	Formula flattening and per-entry validation (a <code>+</code> sum nested in a list is rejected), the one-parented-I rule plus transform hoisting in one pass. An option another term takes is refused by Python's own argument binding, as a <code>TypeError</code> .
(<code>_check_node</code> , <code>_kahn_sort</code>)	The stages behind <code>validate_and_sort</code> : parents exist, then each term's check (the VC treatment and centering rules), then edge ownership through <code>edge_parents</code> ; a term's own shape (arity, option values) and a node's own (ordinal levels, the transform) are checked when they are built. Kahn's sort emits ready nodes in sorted batches, so the order is deterministic.

transforms.py — the monotone map h and the ordinal transform

Name	Role
StandardLogistic	The TRAM base distribution: <code>log_prob</code> , <code>sample</code> (generator-aware), <code>icdf</code> .
BernsteinUT	Bernstein-polynomial transform, the default. Linear tail extrapolation follows the boundary derivative. <code>marginal_init_theta(column)</code> gives the start <code>marginal_init=True</code> applies.
SplineUT	Monotone rational-quadratic spline. Tails extrapolate with a fixed slope (zuko-upstream.md).
AffineUT	Monotone affine transform: the node-conditional is a logistic GLM.
<code>make_univariate_transform()</code>	Transform registry: <code>name</code> $\rightarrow$ transform instance.
<code>ordinal_cutpoints()</code>	Unconstrained $(n, K-1) \rightarrow$ increasing cutpoints with $\pm\infty$ ends.
<code>ordinal_log_prob()</code>	$\log P(Y = y)$, computed in log space (model.md).
<code>ordinal_pmf()</code> / <code>ordinal_sample()</code> / <code>ordinal_abduct()</code>	Class probabilities / latent $\rightarrow$ level / truncated-logistic latent recovery (Pearl step 1) for ordinal nodes.
<code>ordinal_marginal_init_theta()</code>	Cutpoint start that matches the empirical class frequencies (<code>marginal_init=True</code>).
<code>ordinal_bounds()</code>	The shifted cutpoint interval of each observed level. <code>scores.py</code> reads it for the latent-scale derivative.
<code>(_ScaledUT, _loglmexp)</code>	Quantile pre-scaling base class, whose inverse is zuko's with its closed-form tail. Stable $\log(1 - e^x)$.

flow.py — the model

Name	Role
CausalFlowDAG	The flow: one Node per variable in topological order. Construction seeds the weights.
<code>calibrate()</code>	The data-dependent state of every term, taken once from the training rows; <code>marginal_init=True</code> also starts every simple intercept at its marginal (fitting.md).
<code>fit()</code>	Joint maximum likelihood by one minibatch Adam loop, with <code>validation_data=/validation_split=</code> , <code>verbose=</code> , <code>optimizer=</code> and <code>callbacks=</code> ; history holds train, val and lr per epoch. A second call continues training. The mechanics are in fitting.md.
<code>fit_classical()</code>	Float64 full-batch L-BFGS for all-ls specs; refuses flexible specs.
<code>sample()</code>	Observational, interventional (<code>do=</code> , graph mutilation) and counterfactual (<code>u=</code>) sampling.
<code>abduct()</code>	Pearl step 1: recover the latents.
<code>pmf()</code>	Analytic class probabilities of an ordinal node, with <code>do=</code> overrides.
<code>density()</code>	Analytic conditional density of a continuous node on a grid, with <code>do=</code> overrides — the continuous counterpart of <code>pmf</code> .
<code>log_prob()</code> / <code>node_negative_log_prob()</code> (alias <code>nll</code>)	Joint per-row log-likelihood, or a <code>nodes=</code> subset (exact, and the log-space way to get one node's conditional likelihood per row) / mean per-node NLL diagnostic.
<code>node_log_prob()</code>	The per-node decomposition everything trains and evaluates through.
<code>varying_coef()</code>	Closed-form read-out $\beta(x)$ of a fitted VC term. Deterministic, y-free.
<code>scores()</code> / <code>effect_modifier_scan()</code>	Analytic per-observation scores and the CUSUM modifier scan (delegate to <code>scores.py</code>).

Name	Role
<code>intercept_contributions()</code>	Post-hoc GAM-style decomposition of a complex intercept into mean-centered per-term parts.
<code>ls_coefficients()</code>	The per-node linear-shift weights — the interpretable coefficients.
<code>design_matrix()</code>	Parent encoding as a DataFrame (<code>drop_first=</code> gives the classical statsmodels/polr design).
<code>to_matrix()</code>	The labeled meta-adjacency matrix of term tags.
<code>save() / load()</code>	Checkpoints with history and provenance (version, time, device). <code>load</code> requires a complete checkpoint and fails loudly otherwise.
<code>(_node, _features, _tensorize, _generator, _dtype, _init_linear)</code>	Node lookup with one shared error; parent encoding through <code>Node.encode</code> (continuous raw, ordinal one-hot); <code>_tensorize(df, cols=None)</code> for any column subset, checking every ordinal column against its levels on the way in; seeded-generator and dtype plumbing.
<code>(_check_side_columns, _propensity, _side_feats, _query_side_columns, _recenter_vc)</code>	The generic side-column plumbing (each term names/validates/recomputes its own columns via the <code>ShiftModule</code> hooks) plus the binary propensity fit and the post-fit finalize loop.

modules.py — the term modules

There is one module class per term, and the term class holds it as module (`ComplexShift.module` is `ComplexShiftModule`); the module holds the term’s network and owns the runtime hooks (`architecture.md`).

The default architectures replicate the PyTorch reference `buehlpa/TramDag`, file `tram_models.py`, so a fitted model stays comparable to it. They are not the paper’s nets, which `paper-replication.md` lists per experiment.

Name	Role
<code>intercept_module()</code>	The one factory: an I term becomes the free theta, one joint net, or one net per parent (<code>allow_interaction=False</code>).
<code>feat_width()</code>	Total encoded width of a parent set: levels per ordinal parent, 1 per continuous one.
<code>ShiftModule / InterceptModule</code>	The behavior hooks a term module owns: <code>__init__(term, spec)</code> (builds the net, sets key and parents), <code>shift_value/theta_value</code> , <code>post_init</code> , regularizer, post-fit finalize, <code>score_columns</code> , the side-input contract.
<code>LinearShiftModule</code>	LS: <code>Linear(n, 1, bias=False)</code> . <code>.weight</code> is the interpretable coefficient; no bias because the intercept slot owns the constant.
<code>ComplexShiftModule</code>	CS: an NN to one shift value.
<code>VaryingCoefficientModule</code>	VC: $\beta_0 + b_{\Theta}(\text{mod})$ with the L2 hook <code>l2()</code> ; <code>beta()</code> evaluates the effect and <code>recenter()</code> re-splits <code>beta0/b_theta</code> after training (<code>varying-coefficients.md</code>). regressor is both the forward regressor and the <code>beta0</code> score.
<code>SimpleInterceptModule / ComplexInterceptModule / AdditiveInterceptModule</code>	The intercept slot: free theta (<code>I()</code>), one joint net from the parent features to the transform parameters, or one net per parent summed in coefficient space. The additive one holds its nets in nets.
<code>(_nn)</code>	The one NN builder: a stack of the given units with the term’s activation (optional <code>batch_norm</code> before it), then a bias-free output layer.

Name	Role
(<code>_InputTransform</code>)	One term's frozen network-input transform (minmax / standardize / callable over frozen train columns).

nodes.py — the node model

Name	Role
Node	One sub-model per variable: builds its intercept and shift terms through each term's module; <code>theta_shift()</code> sums the terms' <code>shift_values</code> (plain shifts first, then VC); <code>net_input()</code> feeds every term network, <code>input_transform</code> applied.
Node <code>.log_prob</code> / <code>.sample</code> / <code>.abduct</code> / <code>.marginal_theta</code> / <code>.encode</code>	The continuous-vs-ordinal branches and the parent encoding (architecture.md).

fitting.py — FitMixin

Name	Role
<code>fit()</code> / <code>fit_classical()</code> (<code>_split_validation</code> , <code>_normalize_callbacks</code> , <code>_check_fit_sizes</code> , <code>_learning_rates</code> , <code>_log_epoch</code> , <code>_fit_epoch</code> , <code>_FnCallback</code>)	Defined here once, methods of the flow via the mixin. The loop plumbing: Keras-shaped validation split, callback normalization, the validation pass, the rate record, verbose printing.

readouts.py — ReadoutsMixin

Name	Role
<code>shift_curve()</code>	One fitted shift term on a 1-D grid, through the term's own <code>shift_value</code> .
the read-out methods	<code>varying_coef</code> , <code>ls_coefficients</code> , <code>to_matrix</code> , <code>intercept_contributions</code> , <code>design_matrix</code> — defined here once, methods of the flow via the mixin.

scores.py — effect-modifier detection

Name	Role
<code>node_scores()</code>	Per-observation scores of the interpretable shift coefficients (scores.md).
<code>effect_modifier_scan()</code> <code>sup_bb_pvalue()</code>	The fluctuation scan over candidate modifiers (scores.md). $P(\sup B > \text{stat})$ for a Brownian bridge B , the Kolmogorov series.
(<code>_dl_ds</code> , <code>CRIT_5PCT</code>)	Closed-form latent-scale derivative and the 5 % critical value. The per-term columns come from each term's <code>score_columns</code> hook.

callbacks.py — the shipped fit callbacks

Name	Role
EarlyStopping	Best-validation weights, restored at fit end (restore_best=False keeps the final ones); patience= also stops the fit.
PerNodeEarlyStopping	Per-node early stopping: a node freezes after patience flat epochs and loads its best-epoch weights back (restore_best=False keeps the last ones); lr_patience= decays the rate on the way. Stops the fit once every node froze and records best_epoch and frozen as {node: epoch}. step(flow, nll, opt, epoch) for a hand-driven loop.
per_node_adam()	Adam with one node-tagged parameter group per node — the optimizer PerNodeEarlyStopping needs.

plots.py — the figures (matplotlib optional: tramdag[plots])

Name	Role
plot_dag()	The labelled DAG of a spec or flow: layered left to right, ellipses for continuous and rounded boxes for ordinal nodes, every edge drawn by the term that owns it (LS / CS / CI / VC + modifiers, joint for a multi-parent net). Exported as tramdag.plot_dag.
plot_marginals()	Observed vs sampled marginal per node, one panel each.
plot_training()	Summed train/val NLL per epoch, with a dashed mark per frozen= entry.
(_layout, _term_edges)	Longest-path layers with one barycenter sweep; the edge list with the VC treatment/modifier split. matplotlib is imported on the first call, never at package import.

What is *not* in the package

Three groups of files are research code:

- the SCM generators
- the frozen datasets
- the replication scripts

They live in experiments/, outside the installed package; experiments/README.md describes them and tests/README.md what the framework tests measure instead.

Where every training hyperparameter lives

Everything that shapes a fit is either a keyword you pass or a documented default you can read at the call site. Nothing numeric is buried.

Knob	Where	Default
learning rate, batch size validation, progress	fit() fit(validation_data=, validation_split=, verbose=)	1e-2 / 512 (in-repo callers state them explicitly anyway) validation off, verbose=0

Knob	Where	Default
schedules, early stopping	<code>fit(optimizer=, callbacks=)</code>	<code>tramdag.callbacks</code> ships <code>EarlyStopping</code> , <code>PerNodeEarlyStopping</code> ; anything else is torch's <code>lr_scheduler</code> and a few lines of callback (<code>fitting.md</code>)
calibrated init	<code>fit(marginal_init=, calibrate(marginal_init=)</code>	False: zuko's zero start; applied once, on the calibrating fit
VC stage-1 propensities	the training-frame column <code>VC(center=) names</code>	required for a centered VC term (<code>varying-coefficients.md</code>)
VC penalty and centering	<code>VC(penalty=, center=)</code>	1.0 / False (<code>center="col"</code> names the propensity column)
L-BFGS budget	<code>fit_classical(max_iter=, history_size=)</code>	400 / 50; torch's <code>tolerance_change</code> is 1e-9 and <code>tolerance_grad</code> is off — one full-batch run, no chunks; converged also needs <code>GRAD_TOL</code> (1e-2)
training budget	<code>fit(epochs=)</code>	required (<code>fitting.md</code>)
network widths	<code>units=</code> on I/CS/VC	(8, 8) / (64, 128, 64) — parity with the PyTorch reference's default classes; VC's (16,) has no counterpart there and comes from the recovery measurement
activation	<code>activation=</code> on I/CS/VC	"relu" (the reference default classes); "sigmoid" and "tanh" are the paper's
batch norm	<code>batch_norm=</code> on I/CS/VC	False — neither reference uses it. True puts a <code>BatchNorm1d</code> between each hidden layer and its activation, so the fit needs more than one row per batch and inference needs <code>eval()</code> mode (fit and load leave the flow there)
transform class	<code>I(transform=, **kwargs)</code> (extra kwargs go to the transform class)	"bernstein", <code>n_coeffs=20</code> unconstrained coefficients (<code>zuko-upstream.md</code> on the order they give); <code>spline bins=8</code> , zuko's NSF default; the domain is fixed at [-5, 5], <code>transforms.BOUND</code>
shuffling / weight init	<code>fit(seed=)</code> / <code>CausalFlowDAG(seed=)</code>	init happens at construction — the constructor seed is the reproducibility knob
weight init	<code>CausalFlowDAG(init=)</code>	"torch" (<code>nn.Linear</code> Kaiming-uniform); "glorot" = Keras Dense default, glorot-uniform weights and zero biases — the paper's reference; decisive under its full-batch protocol
network inputs	CI/CS/VC(<code>input_transform=</code>)	None, raw parents; "minmax", "standardize" or a callable <code>fn(x, train)</code> (<code>model.md</code>)

Upstream candidates for zuko

This document lists what `tramdag` works around in `zuko` (1.6.0). The list is ranked by value against effort, as candidate upstream PRs or issues. `tramdag` uses `zuko` in one place only. It uses these three transforms from `zuko.transforms`:

- `BernsteinTransform`
- `MonotonicRQSTransform`
- `MonotonicAffineTransform`

The three wrappers in `src/tramdag/transforms.py` hold them. The flow machinery, the base distribution and the DAG structure are `tramdag`'s own. The suggested order of attack is $4 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 5$.

1. Analytic call_and_ladj for BernsteinTransform

BernsteinTransform inherits MonotonicTransform.call_and_ladj. That method gets the Jacobian by torch.autograd.grad in the forward pass. It therefore roughly doubles the training graph.

It also makes the per-node loss un-torch.compile-able. Autograd in the forward pass gives a double backward, and aot_autograd does not support one. Therefore the op-count axis stays closed in eager mode.

The derivative is closed form: $f'(x) = \text{order} \cdot \Delta\theta$ against the order-1 basis. zuko already computes it in _offset_and_slope for the tail slopes. An override is therefore ~25 lines and needs no API change. MonotonicRQSTransform already has such an override. tramdag gets a free speedup and the torch.compile axis back.

2. Learnable/linear tails for MonotonicRQSTransform

zuko pads the knot derivatives with $\exp(0)=1$. It extrapolates as the identity outside $[-B, B]$. Therefore the tail slope stays fixed at 1 for every θ . This is the structural reason why spline consistently trails bernstein here, because ~10% of the data sits beyond the 5%/95% pre-scaling range.

Upstream, zuko can accept boundary derivatives of shape $(*, K+1)$, or it can take an opt-in tails="linear". Identity tails are deliberate in the NSF design. The framing must therefore stay back-compatible. This change will delete the caveats and make spline a first-class transform choice.

3. Public inverse of _constrain_theta

BernsteinUT.marginal_init_theta inverts zuko's *private* _constrain_theta (cumsum-of-softplus) in closed form. It hard-codes two internal details of that function:

- the $\log(2) \cdot n/2$ centering
- the tying of the first two and the last two diffs

Any upstream reparametrization therefore breaks the calibrated start silently. A public unconstrain_theta(theta) that takes the target control points is ~15 lines upstream. It removes the coupling to these private details. The framing is "identity/linear initialization support", a standard flow trick.

4. Docstring fix: the Bernstein θ -shape off-by-one

zuko's docstring says that θ has shape $(*, M-2)$ for a degree-M polynomial. n unconstrained coefficients become $n+2$ control points, which is degree $n+1$. The correct claim is therefore $(*, M-1)$. This off-by-one is a real replication trap: order 21 against the paper's len_theta=20, which is order 19. The PR is trivial and an accept is near-certain.

5. Logistic in zuko.distributions

Neither zuko nor torch ships a Logistic distribution. tramdag therefore hand-rolls StandardLogistic in ~50 lines, with log_prob, sample and icdf. Logistic latents are standard in TRAM-DAGs and in discrete flows, and zuko has a precedent in GeneralizedNormal. But zuko can point at TransformedUniform(SigmoidTransform().inv) instead. tramdag keeps its _U_EPS clamp locally in both cases. This candidate has the lowest value of the five.

Anti-candidates (look upstreamable, are not)

- **Quantile pre-scaling and _ScaledUT** — this is a modeling choice. zuko's ComposedTransform with MonotonicAffineTransform already composes it.

- **The ordinal ordered-logit transform and log-space ordinal_log_prob** — this transform is not a bijection. It stays outside zuko’s flow scope. It is torch.distributions material, if anywhere.
- **LS, CS, CI and VC terms, the marginal-init policy, propensity centering and scores** — these are TRAM-DAG semantics. zuko’s MaskedMLP already supports arbitrary adjacency. tramdag does not use it, because it needs per-term interpretability.
- **The spline slope knob and the Bernstein eps knob** — zuko already exposes both upstream.

Additive vs joint complex intercept — interpreting per-parent effects

A complex intercept groups its parents jointly, `CI("x1", "x2")`, or additively, `CI("x1", "x2", allow_interaction=False)`. `docs/model.md` states the rule, and `docs/interpretation.md` why the additive form needs `flow.intercept_contributions(data, node)` to be read.

This notebook fits both models and shows the interpretability difference. The punchline is perhaps surprising: the two models fit the **data** almost identically — the difference is **structural**, visible only in parameter space, and that is precisely what `intercept_contributions` surfaces.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```
from tramdag import CI, CausalFlowDAG, ContinuousNode
from tramdag.callbacks import EarlyStopping
```

The data

$x_1, x_2 \sim U(-2, 2)$ and $x_3 = x_1 + x_2 + \gamma x_1 x_2 + \frac{1}{2} \varepsilon$, $\varepsilon \sim \text{Logistic}(0, 1)$ — a genuine $x_1 \cdot x_2$ interaction.

`GAMMA = 0.6`

```
def simulate(n, seed):
    rng = np.random.default_rng(seed)
    x1 = rng.uniform(-2, 2, n)
    x2 = rng.uniform(-2, 2, n)
    eps = rng.logistic(0, 1, n)
    x3 = x1 + x2 + GAMMA * x1 * x2 + 0.5 * eps
    return pd.DataFrame({"x1": x1, "x2": x2, "x3": x3})
```

```
train = simulate(4000, seed=1)
val = simulate(1000, seed=2)
train.head()
```

x1

x2

x3

0

0.047286

```

0.755745
1.239427
1
1.801855
0.126365
0.853817
2
-1.423362
0.970406
-0.294179
3
1.794598
1.590401
4.880817
4
-0.752674
-0.168717
-0.689205

```

Fit both models

Same data, same node — only the grouping of the two I parents differs.

```

def make_flow(joint: bool):
    spec = {
        "x1": ContinuousNode(),
        "x2": ContinuousNode(),
        "x3": ContinuousNode(
            CI("x1", "x2") if joint else CI("x1", "x2", allow_interaction=False)
        ),
    }
    return CausalFlowDAG(spec, seed=0)

```

```

flow_joint = make_flow(joint=True)
flow_add = make_flow(joint=False)

```

```

for f in (flow_joint, flow_add):
    f.fit(
        train,
        epochs=1200,
        learning_rate=1e-2,
        validation_data=val,
        callbacks=EarlyStopping(), # keep the best-validation weights
    )

```

Both fit the data well; the joint model is only marginally better on held-out

```

# likelihood. A flexible additive Bernstein intercept can *mimic* a lot of
# apparent interaction through its nonlinear transform, so you generally **cannot
# tell the two apart from the fitted distribution** – see the note below.
print("val NLL joint :", round(sum(flow_joint.nll(val).values()), 4))
print("val NLL additive:", round(sum(flow_add.nll(val).values()), 4))

val NLL joint : 4.1711
val NLL additive: 4.202

```

intercept_contributions — the exact, centered decomposition

It returns each I-term’s mean-centered (sum-to-zero) contribution to the transform parameters theta, plus the absorbed baseline. The centering is over the rows of the df you pass, and the decomposition is exact.

```

res_add = flow_add.intercept_contributions(train, "x3")
res_joint = flow_joint.intercept_contributions(train, "x3")

print("additive terms :", list(res_add["contributions"])) # 'x1', 'x2' – separable
print("joint terms    :", list(res_joint["contributions"])) # 'x1+x2' – inseparable

# the decomposition is exact (baseline + contributions == theta, pinned by the
# test suite); the centering is easy to see here: every column mean is ~0
print(
    "per-term column means (≈0) :",
    {k: float(np.abs(v.mean(0)).max()) for k, v in res_add["contributions"].items()},
)

additive terms : ['x1', 'x2']
joint terms    : ['x1+x2']
per-term column means (≈0) : {'x1': 3.5154818078808603e-07, 'x2': 6.728172365910723e-07}

```

The structural difference: separable curves vs an entangled cloud

Pick the transform coefficient that varies most across the data and plot each term’s centered contribution to it against a parent value.

- **Additive** (net₁ depends only on x₁): its contribution to any coefficient is a deterministic 1-D function of x₁ — the points collapse to a **clean curve** (same for x₂). These curves *are* the per-parent partial effects: well-defined because the structure is separable.
- **Joint** (one network over both parents): the single x₁+x₂ component depends on *both*, so plotted against x₁ alone it is a **cloud** whose height is explained by the hidden x₂ (color). There is nothing to read off per parent.

```

c1 = res_add["contributions"]
c2 = res_add["contributions"]
cj = res_joint["contributions"]
k = int((c1.var(0) + c2.var(0)).argmax()) # most-varying coefficient
x1v, x2v = train["x1"].values, train["x2"].values

fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), sharey=True)
o1, o2 = np.argsort(x1v), np.argsort(x2v)
axes[0].plot(x1v[o1], c1[o1, k], color="#1b9e77", lw=2.5, label="net(x1) vs x1")
axes[0].plot(x2v[o2], c2[o2, k], color="#d95f02", lw=2.5, label="net(x2) vs x2")
axes[0].axhline(0, color="0.7", lw=0.8)
axes[0].set_title("additive – separable per-parent curves")
axes[0].set_xlabel("parent value")

```

```

axes[0].set_ylabel(f"centered contribution to theta[{k}]")
axes[0].legend()

sc = axes[1].scatter(x1v, cj[:, k], c=x2v, s=8, alpha=0.6, cmap="coolwarm")
axes[1].axhline(0, color="0.7", lw=0.8)
axes[1].set_title("joint — vs x1 is a cloud (height set by x2)")
axes[1].set_xlabel("x1")
fig.colorbar(sc, ax=axes[1], label="x2")
fig.tight_layout()
plt.show()

```

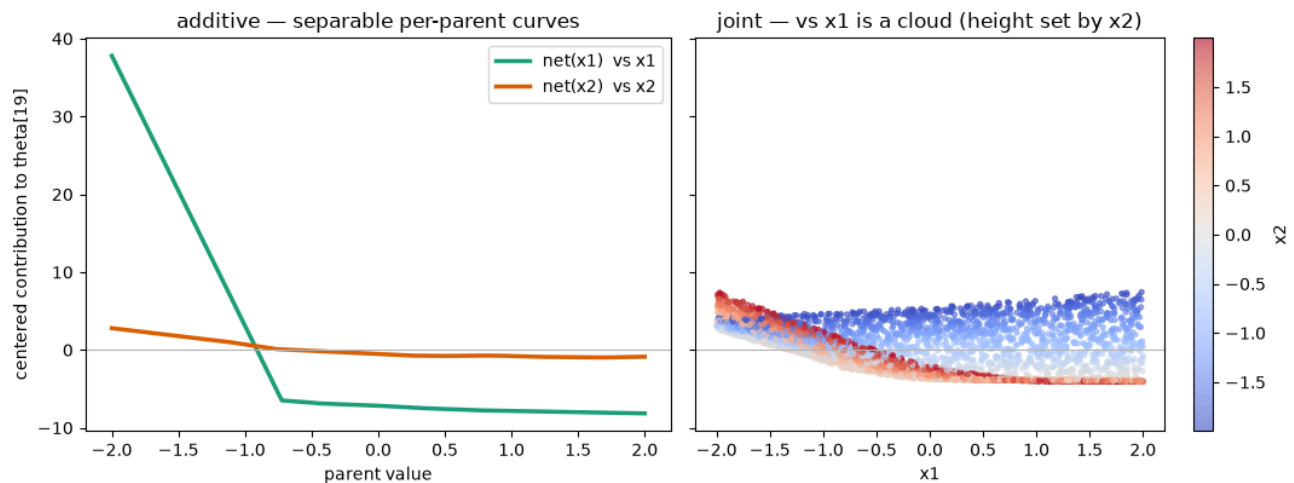


Figure 1: png

Takeaway

you want...	use	what you get
a per-parent partial-effect plot ("what does x1 do?")	additive CI("x1", "x2", allow_interaction=False)	intercept_contributions → exact, mean-centered, separable components
interactions between parents in the transform	joint CI("x1", "x2")	one entangled network — flexible, not separable

The choice is about *interpretability*, not correctness: as the near-equal NLLs show, the fitted distribution alone does not distinguish them.

Caveat: contributions live in the transform's **unconstrained** parameter space (where the additive terms are summed, before the monotonicity constraint), so they are exact partial effects on the parameters but not, in general, an additive split of the curve h itself.

Classical fitting of all-1s TRAM-DAGs (fit_classical)

When **every edge of a TRAM-DAG is a linear shift (LS)**, each node-conditional is a classical model

- an ordered-logit / proportional-odds model for ordinal nodes
- a Colr model (R's `tram::Colr`) for continuous ones.

For such a model `flow.fit_classical()` is the optimizer: deterministic full-batch L-BFGS whose report says when it reached the maximum-likelihood estimate (`docs/fitting.md`). This notebook checks that against `statsmodels` and `R`.

Every `R` reference printed below is hard-coded from a real fit, and every one of those fits lives in `notebooks/classical_fit_tram_dag.R`. Run `Rscript notebooks/classical_fit_tram_dag.R` from the repo root to re-check them; it needs `MASS` (ships with `R`) and `tram`.

```
import time
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from statsmodels.miscmodels.ordinal_model import OrderedModel

from tramdag import LS, SI, CausalFlowDAG, ContinuousNode, OrdinalNode
from tramdag.callbacks import PerNodeEarlyStopping, per_node_adam

# repo-relative data, whether the notebook runs from the repo root or notebooks/
HERE = [Path.cwd(), *Path.cwd().parents]
REPO = next(p for p in HERE if (p / "pyproject.toml").exists())
DATA = REPO / "experiments" / "data"
NB_DATA = REPO / "notebooks" / "data"

def claim(text, value, bound, fmt="{:.2e}"):
    """Assert a computed quantity stays inside its bound, and print both.

    Every agreement this notebook asserts goes through here, so no number is
    typed into a markdown cell where it can drift away from the code. The
    prose says what is being checked and why the bound is what it is; the
    bound is the only number in the source.

    A bound sits at roughly 1.5 to 4 times the measured value, which is the
    band that ``experiments/ground_truth/*.json`` already uses. Tighter than
    that and another machine trips it; wider and it stops meaning anything.
    """
    ok = "ok " if value <= bound else "FAIL"
    print(f"{ok}{text}: {fmt.format(value)} <= {fmt.format(bound)}")
    assert value <= bound, f"{text}: {fmt.format(value)} exceeds {fmt.format(bound)}"
```

0. The zeroth example — logistic regression on `MASS::birthwt`

Before any DAG, take the model everyone already knows, on a dataset every `R` user already has. A **two-level** `OrdinalNode` whose terms are all `LS` is logistic regression — not an analogue of it, the same model. The ordinal transform is

$$P(Y \leq 0 \mid x) = \sigma(\theta_0 - \sum_p w_p x_p),$$

and with two levels there is a single cutpoint θ_0 , so

$$\text{logit } P(Y = 1 \mid x) = -\theta_0 + \sum_p w_p x_p.$$

The shift weights **are** the logistic-regression coefficients; the intercept is **minus** the cutpoint, because an ordinal node *subtracts* its shift (docs/notation.md). Section 2 is this same picture with $K - 1$ cutpoints instead of one.

The data is birthwt from **MASS**, the low-birth-weight study of Hosmer & Lemeshow: 189 births, outcome low (birth weight under 2.5 kg), predictors age (mother's age), lwt (mother's weight at last period, lbs) and smoke (smoking during pregnancy). notebooks/data/birthwt.csv is those four columns exported verbatim from MASS. Please note that this is not a **causal** model.

The R you can copy-paste

birthwt ships with MASS, so nothing needs downloading — paste this into any R session and you have the reference fit:

```
library(MASS)
m <- glm(low ~ age + lwt + smoke, data = birthwt, family = binomial)
coef(m)
logLik(m)
```

which prints

```
(Intercept)          age          lwt          smoke
  1.36822527 -0.03899458 -0.01213854  0.67076374
'log Lik.' -111.4396765 (df=4)
```

(R 4.2.3, MASS 7.3-58.2.) Those four numbers are hard-coded below as R_GLM, so the notebook can show the three-way comparison without an R installation.

```
R_GLM = { # coef(glm(low ~ age + lwt + smoke, birthwt, family = binomial))
  "intercept": 1.3682252685,
  "age": -0.0389945827,
  "lwt": -0.0121385423,
  "smoke": 0.6707637407,
}
R_LOGLIK = -111.43967649
```

```
bw = pd.read_csv(NB_DATA / "birthwt.csv")
print(
  bw.head(3).to_string(index=False),
  f"\n\n{len(bw)} births, {bw['low'].sum()} of them low birth weight",
)
```

```
low  age  lwt  smoke  bwt
0    19  182     0  2523
0    33  155     0  2551
0    20  105     1  2557
```

189 births, 59 of them low birth weight

The spec. low is the outcome; smoke is a binary **parent**, so it is an ordinal node too. age and lwt are source nodes — the flow models their marginals as well, which a plain glm does not. That is a nuisance here, not a feature, so they get the cheapest transform (SI(transform="affine")), and it cannot disturb the outcome node in any case: the joint NLL decomposes per node, so the low node's gradient never sees the covariate marginals.

```
spec_bw = {
  "age": ContinuousNode(SI(transform="affine")),
  "lwt": ContinuousNode(SI(transform="affine")),
  "smoke": OrdinalNode(2),
  "low": OrdinalNode(2, LS("age") + LS("lwt") + LS("smoke")),
}
```

```

}

flow_bw = CausalFlowDAG(spec_bw, seed=0)
flow_bw.fit_classical(bw)
{'converged': True,
 'stop_reason': 'tolerance',
 'n_iter': 64,
 'final_nll': 9.137612929338468,
 'grad_norm': 0.0001517562203257858,
 'seconds': 0.907368369999972,
 'coefficients': {'low': {'age': array([-0.03899488]),
 'lwt': array([-0.01213841]),
 'smoke': array([-0.13497243,  0.53579548])}}}

```

Now read the coefficients back out. age and lwt are continuous parents, so they enter raw and their weights compare directly. smoke is ordinal, so it enters **one-hot over both levels**.

Careful — different coding from R. R gives a binary predictor one column; the flow gives it two, w_0 and w_1 , identified only through their difference (docs/interpretation.md), and that difference is what R's single smoke coefficient means. Section 2 uses the same rule for T.

The second column also moves the intercept. With $s \in \{0, 1\}$ we have $\mathbf{1}[s = 0] = 1 - s$, so the one-hot pair collapses to

$$w_0 \mathbf{1}[s = 0] + w_1 \mathbf{1}[s = 1] = w_0 + (w_1 - w_0) s,$$

and the node reads

$$\text{logit } P(\text{low} = 1) = (-\theta_0 + w_0) + w_{\text{age}} \text{age} + w_{\text{lwt}} \text{lwt} + (w_1 - w_0) s.$$

The first bracket is R's (Intercept) — it is $-\theta_0 + w_0$, not $-\theta_0$ — and the last is R's smoke.

```

import statsmodels.formula.api as smf # noqa: E402

from tramdag.transforms import ordinal_cutpoints # noqa: E402

# the same formula as the R call above, and the same names in the output
res_bw = smf.logit("low ~ age + lwt + smoke", data=bw).fit(dispen=False)

w = flow_bw.ls_coefficients()["low"]
with torch.no_grad(): # cutpoints are [-inf, theta_0, +inf]; take the finite one
    theta_0 = float(ordinal_cutpoints(flow_bw.nodes["low"].intercept(1))[0, 1])

rows_bw = [
    # -theta_0 + w_smoke[0]: the one-hot level-0 column is part of the intercept
    (
        "intercept",
        -theta_0 + float(w["smoke"]),
        res_bw.params["Intercept"],
        R_GLM["intercept"],
    ),
    ("age", float(w["age"]), res_bw.params["age"], R_GLM["age"]),
    ("lwt", float(w["lwt"]), res_bw.params["lwt"], R_GLM["lwt"]),
    (
        "smoke",

```

```

        float(w"smoke" - w"smoke"),
        res_bw.params["smoke"],
        R_GLM["smoke"],
    ),
]
print(f"{'':<11}{'fit_classical':>14}{'sm.Logit':>10}{'R glm':>10}{'max|diff|':>11}")
for name, flow_value, sm_value, r_value in rows_bw:
    spread = max(abs(flow_value - sm_value), abs(flow_value - r_value))
    print(
        f"{'name':<11}{'flow_value':>14.4f}{'sm_value':>10.4f}{'r_value':>10.4f}{'spread':>11.2e}"
    )

```

	fit_classical	sm.Logit	R glm	max diff
intercept	1.3682	1.3682	1.3682	6.46e-06
age	-0.0390	-0.0390	-0.0390	2.96e-07
lwt	-0.0121	-0.0121	-0.0121	1.32e-07
smoke	0.6708	0.6708	0.6708	4.16e-06

Three optimizers — L-BFGS on the flow, statsmodels' Newton solver, R's IRLS — land on the same maximum likelihood, because there is only one. The agreement is not only in the parameters: the log-likelihood of the low node and R's logLik(m) are the same number, and flow.pmf reproduces glm's fitted probabilities row by row.

```

# nll() is the *mean* NLL per node; times n and negated it is the log-likelihood
ll_flow = -flow_bw.nll(bw)["low"] * len(bw)
p_flow = flow_bw.pmf(bw, node="low")[:, 1]
p_classical = res_bw.predict(bw).values
print(f"log-likelihood    flow {ll_flow:.6f}    R glm {R_LOGLIK:.6f}")
print(
    f"max |P_flow(low=1) - P_glm(low=1)| over {len(bw)} rows: "
    f"{np.abs(p_flow - p_classical).max():.2e}"
)
log-likelihood    flow -111.439677    R glm -111.439676
max |P_flow(low=1) - P_glm(low=1)| over 189 rows: 3.32e-06

```

1. Continuous case

Section 0 modelled low, the *dichotomized* birth weight. birthwt also carries the number it was cut from — bwt, the weight in grams — so the same three predictors can be fitted against a continuous outcome. That is a Colr model, R's `tram::Colr bwt ~ age + lwt + smoke` No way to model this with glm.

```

R_COLR_BWT = { # Colr(bwt ~ age + lwt + smoke, order = 21); R 4.2.3 / tram 1.0.4
    "age": -0.021971,
    "lwt": -0.010233,
    "smoke": 0.668698,
    "loglik": -1499.6678,
}

spec_bwt = {
    "age": ContinuousNode(SI(transform="affine")), # nuisance marginals
    "lwt": ContinuousNode(SI(transform="affine")),
    "smoke": OrdinalNode(2),
    "bwt": ContinuousNode(
        SI(n_coeffs=20) + LS("age") + LS("lwt") + LS("smoke") # degree 21
    ),
}

```

```

flow_bwt = CausalFlowDAG(spec_bwt, seed=0)
flow_bwt.fit_classical(bw, max_iter=800)

w = flow_bwt.ls_coefficients()["bwt"]
fitted = {
    "age": w"age",
    "lwt": w"lwt",
    "smoke": w"smoke" - w"smoke", # one-hot: read the difference
    "loglik": -flow_bwt.nll(bw)["bwt"] * len(bw),
}

print(f"{'':<9}{'age':>9}{'lwt':>9}{'smoke':>9}{'logLik':>11}")
for name, r in [("flow", fitted), ("R Colr", R_COLR_BWT)]:
    print(
        f"{name:<9}{r['age']:>9.4f}{r['lwt']:>9.4f}{r['smoke']:>9.4f}"
        f"{r['loglik']:>11.2f}"
    )
diff = {k: abs(fitted[k] - R_COLR_BWT[k]) for k in fitted}
print(
    f"{'|diff|':<9}{diff['age']:>9.4f}{diff['lwt']:>9.4f}{diff['smoke']:>9.4f}"
    f"{diff['loglik']:>11.2f}"
)

```

	age	lwt	smoke	logLik
flow	-0.0223	-0.0103	0.6641	-1500.55
R Colr	-0.0220	-0.0102	0.6687	-1499.67
diff	0.0004	0.0000	0.0046	0.88

The largest *absolute* difference is in smoke, but that only reflects its coefficient being some thirty times bigger than the others; as a fraction of the estimate it is the smallest of the three. Neither comparison is the useful one. A difference matters when it is large relative to the **sampling uncertainty** of the estimate — and that is a quantity we have not computed yet.

Standard errors and confidence intervals

The sampling distribution of the maximum likelihood estimator is

$$\hat{\theta} \sim N(\theta, I^{-1})$$

where I is the Fisher information matrix. Since θ is unknown we evaluate I at $\hat{\theta}$, and read the standard errors off the diagonal of I^{-1} . For a contrast c , we read off $\sqrt{c^\top I^{-1} c}$.

The **observed** Fisher information is the Hessian of the *negative* log-likelihood at the MLE,

$$I_{ab} = \frac{\partial^2(-\ell)}{\partial \theta_a \partial \theta_b} \Big|_{\hat{\theta}},$$

which autograd gives directly: one backward pass produces the gradient with the graph retained, then one further backward pass per component produces a row of I .

Here I is singular, so I^{-1} does not exist. It is symmetric, so it has an orthonormal eigendecomposition, and the **pseudo-inverse** inverts only the directions that carry curvature:

$$I = \sum_{k=1}^P \lambda_k e_k e_k^\top \quad \Rightarrow \quad I^+ = \sum_{k: \lambda_k > \tau} \frac{1}{\lambda_k} e_k e_k^\top, \quad \tau = 10^{-7} \lambda_{\max}.$$

Terms with $\lambda_k \leq \tau$ are dropped rather than inverted. That is right for a contrast which avoids the flat subspace, and the leak column measures exactly how far it fails to:

$$\text{leak}(c) = \max_{k: \lambda_k \leq \tau} |c^\top e_k|.$$

Only interpret rows whose leak is close to zero. The choice of τ is not delicate, because the spectrum splits into a flat block and a curved block with orders of magnitude between them. Any threshold inside that gap gives the same answer, and the cell below measures the gap rather than asserting it.

```
from scipy.stats import norm # noqa: E402

def observed_information(flow, node, data):
    """Give one node's NLL Hessian, the gradient norm, and parameter offsets."""
    flow.double() # second derivatives need float64
    names, params = zip(*flow.nodes[node].named_parameters(), strict=True)
    nll = -flow.node_log_prob(flow._tensorize(data))[node].sum()
    grad = torch.cat(
        [g.reshape(-1) for g in torch.autograd.grad(nll, params, create_graph=True)]
    )
    hess = torch.stack(
        [
            torch.cat(
                [
                    h.reshape(-1)
                    for h in torch.autograd.grad(grad[i], params, retain_graph=True)
                ]
            )
            for i in range(grad.numel())
        ]
    )
    offsets, at = {}, 0
    for name, p in zip(names, params, strict=True):
        offsets[name] = at
        at += p.numel()
    result = hess.detach().numpy(), float(grad.detach().norm()), offsets
    flow.float() # back to the stored precision
    return result

def spectrum_split(evals, tau):
    """Describe the curvature spectrum either side of ``tau``.

    A wide gap between the flat block and the curved block is what makes the
    threshold uncritical, so this reports both edges of the gap and lets the
    caller check it instead of describing it in prose.
    """
    absolute = np.abs(evals)
    flat, curved = absolute[absolute <= tau], absolute[absolute > tau]
```

```

return {
    "n_flat": int(flat.size),
    "flat_max": float(flat.max()) if flat.size else 0.0,
    "curved_min": float(curved.min()) if curved.size else 0.0,
}

def ls_conf_int(flow, node, data, level=0.95):
    """Give Wald intervals for the linear-shift coefficients of one node."""
    hess, grad_norm, offsets = observed_information(flow, node, data)
    evals, evecs = np.linalg.eigh(hess)
    rcond = 1e-7
    tau = rcond * np.abs(evals).max() # one threshold, used for both
    flat = evecs[:, np.abs(evals) <= tau] # the directions without curvature
    cov = np.linalg.pinv(hess, rcond=rcond) # invert only the rest
    z = norm.ppf(0.5 + level / 2)

    rows = []
    for parent, w in flow.ls_coefficients()[node].items():
        c = np.zeros(hess.shape[0])
        key = next(n for n in offsets if n.startswith(f"shifts.{parent}"))
        if len(w) == 1: # continuous parent: one weight
            c[offsets[key]], estimate, term = 1.0, w[0], parent
        else: # ordinal parent: the contrast w[1] - w[0]
            c[offsets[key]], c[offsets[key] + 1] = -1.0, 1.0
            estimate, term = w[1] - w[0], f"{parent} (1 vs 0)"
        se = float(np.sqrt(c @ cov @ c))
        rows.append(
            {
                "term": term,
                "estimate": estimate,
                "se": se,
                "lower": estimate - z * se,
                "upper": estimate + z * se,
                # 0 = the contrast avoids the flat subspace, so the SE is real
                "leak": float(np.abs(c @ flat).max()) if flat.size else 0.0,
            }
        )
    diagnostics = {
        "n_params": hess.shape[0],
        "grad_norm": grad_norm,
        **spectrum_split(evals, tau),
    }
    return pd.DataFrame(rows).set_index("term"), diagnostics

table, diag = ls_conf_int(flow_bwt, "bwt", bw)
print(table.round(6).to_string())
print(
    f"\n{diag['n_params']} parameters, {diag['n_flat']} of them without curvature"
    f" | grad | = {diag['grad_norm']:.1e}"
)
# Colr(bwt ~ age + lwt + smoke, order = 21) on birthwt; R 4.2.3 / tram 1.0.4,
# produced by classical_fit_tram_dag.R. This is the oracle, not the claim.
R_COLR_BWT_SE = {"age": 0.025305, "lwt": 0.004116, "smoke": 0.260131}

print()

```

```

claim(
  "flat_max / curved_min, so smaller is a wider gap",
  diag["flat_max"] / diag["curved_min"],
  1e-4,
)
se_gap = max(
  abs(float(table.loc[t, "se"]) - r)
  for t, r in [("age", R_COLR_BWT_SE["age"]), ("lwt", R_COLR_BWT_SE["lwt"])]
)
claim("max |SE_flow - SE_Colr|, continuous parents", se_gap, 1e-3)

```

	estimate	se	lower	upper	leak
term					
age	-0.022343	0.025264	-0.071861	0.027174	0.0
lwt	-0.010276	0.004113	-0.018337	-0.002216	0.0
smoke (1 vs 0)	0.664074	0.259577	0.155312	1.172835	0.0

24 parameters, 13 of them without curvature |grad| = 3.4e-02

ok flat_max / curved_min, so smaller is a wider gap: 3.73e-07 <= 1.00e-04

ok max |SE_flow - SE_Colr|, continuous parents: 4.05e-05 <= 1.00e-03

The standard errors of the two continuous parents reproduce Colr's, and R's confint() on a Colr fit is Wald as well.

1b. A bimodal DAG — the demo data

The VACA benchmark triangle $x_1 \rightarrow x_2 \rightarrow x_3 \leftarrow x_1$, the bimodal SCM of the demo notebook. We fit an **all-ls** model: each node is a Colr model: a Bernstein baseline with linear shifts.

Note this is an *honest misspecification*: the DGP noise is Gaussian while the TRAM latent is logistic, so the all-ls model is not the true generator — but fit_classical still finds its exact MLE, which is the point here.

if False:

```

def sample_vaca(n, seed):
    """Draw n rows from the VACA bimodal triangle SCM."""
    rng = np.random.default_rng(seed)
    mix, a, b = rng.uniform(size=n), rng.normal(size=n), rng.normal(size=n)
    x1 = np.where(mix < 0.5, -2.0 + np.sqrt(1.5) * a, 1.5 + b)
    x2 = -x1 + rng.normal(size=n)
    x3 = x1 + 0.25 * x2 + rng.normal(size=n)
    return pd.DataFrame({"x1": x1, "x2": x2, "x3": x3})

df = sample_vaca(1000, seed=42)
df.to_csv(NB_DATA / "vaca.csv", index=False)

```

The R you can copy-paste

tram::Colr fits the same model: a Bernstein baseline with linear shifts. The cell above is what wrote notebooks/data/vaca.csv, which is committed, so R reads the identical rows the flow is fitted on.

The two libraries **count the basis differently**, and the comparison is only meaningful at the same polynomial degree: N_COEFFS = 20 below is tram's order = 21, because n_coeffs counts unconstrained coefficients and zuko ties two control points on (docs/zuko-upstream.md).

```
library(tram)
d <- read.csv("notebooks/data/vaca.csv")

m2 <- Colr(x2 ~ x1, data = d, order = 21)
coef(m2)      # -> x1 1.800042
logLik(m2)    # -> -1401.213542

m3 <- Colr(x3 ~ x1 + x2, data = d, order = 21)
coef(m3)      # -> x1 -1.777289 x2 -0.455282
logLik(m3)    # -> -1411.901958
```

The coefficients need **no sign flip**: Colr reports the shift on the same side as the flow.

```
R_COLR = { # Colr(..., order = 21) on vaca.csv; R 4.2.3 / tram 1.0.4
  "x2": {"coef": {"x1": 1.800042}, "loglik": -1401.213542},
  "x3": {"coef": {"x1": -1.777289, "x2": -0.455282}, "loglik": -1411.901958},
}

df = pd.read_csv(NB_DATA / "vaca.csv")
# stated rather than left to the default, for the reason the markdown cell
# above the R block gives
N_COEFFS = 20 # -> Bernstein degree 21, which is tram's `order = 21`

spec_vaca = {
  "x1": ContinuousNode(SI(n_coeffs=N_COEFFS)),
  "x2": ContinuousNode(SI(n_coeffs=N_COEFFS) + LS("x1")),
  "x3": ContinuousNode(SI(n_coeffs=N_COEFFS) + LS("x1") + LS("x2")),
}

flow_c = CausalFlowDAG(spec_vaca, seed=0)
rep = flow_c.fit_classical(df) # logs iters / NLL / time
print(f"\n{' ':<12}{ 'fit_classical':>14}{ 'R Colr':>12}{ '|diff|':>10}")
loglik = {k: -v * len(df) for k, v in flow_c.nll(df).items()} # nll() is a mean
for node, parents in flow_c.ls_coefficients().items():
  for p, w in parents.items():
    c, r = w[0], R_COLRnode[p]
    print(f"{p + ' -> ' + node:<12}{c:>14.4f}{r:>12.4f}{abs(c - r):>10.4f}")
  c, r = loglik[node], R_COLRnode
  print(f"{'logLik ' + node:<12}{c:>14.4f}{r:>12.4f}{abs(c - r):>10.4f}")
```

	fit_classical	R Colr	diff
x1 -> x2	1.7998	1.8000	0.0002
logLik x2	-1401.8717	-1401.2135	0.6581
x1 -> x3	-1.7806	-1.7773	0.0033
x2 -> x3	-0.4550	-0.4553	0.0003
logLik x3	-1410.9678	-1411.9020	0.9341

Why this one is not an exact-MLE check

The coefficients here agree far less closely than in Sections 0 and 2, and neither result is the more correct one. The two libraries place the Bernstein basis differently: the flow puts it on the range_q quantiles with linear tails (docs/model.md), while Colr uses the full range of the data. The shift coefficients survive that difference in relative terms, which is why the ordered logit above finds the same two numbers without any Bernstein basis at all.

```
gaps = [
  (abs(float(w[0]) - R_COLRnode[parent]), abs(R_COLRnode[parent]))
  for node in R_COLR
```

```

    for parent, w in flow_c.ls_coefficients()[node].items()
]
claim("max |coef_flow - coef_Colr|", max(g for g, _ in gaps), 0.01)
claim("max relative coefficient gap", max(g / r for g, r in gaps), 0.02)
ok max |coef_flow - coef_Colr|: 3.32e-03 <= 1.00e-02
ok max relative coefficient gap: 1.87e-03 <= 2.00e-02

```

The fitted conditional density

`flow.density(df, node, grid)` evaluates $p(\text{node} = g \mid \text{pa})$ at each grid value, in closed form from the transform and with no sampling. It is the continuous counterpart of `flow.pmf`.

Two checks are worth making on it, and neither needs an external reference. A density must integrate to one over a wide enough grid, and its mode must track the conditional mean that the linear shifts imply. The first is the stronger check, because it exercises the log-determinant of the transform rather than only its location.

```

grid = np.linspace(df["x3"].min() - 2, df["x3"].max() + 2, 601)
rows = df.iloc[:200]
dens = flow_c.density(rows, "x3", grid) # (200, 601)
mass = np.trapezoid(dens, grid, axis=1)
claim("worst |integral of the density - 1|", float(np.abs(mass - 1.0).max()), 5e-3)

# the mode should move with the parents in the direction the shifts say: the
# flow subtracts on the latent scale, so a positive weight lowers the value
modes = grid[dens.argmax(axis=1)]
shift = -(
    float(flow_c.ls_coefficients()["x3"][0]) * rows["x1"]
    + float(flow_c.ls_coefficients()["x3"][0]) * rows["x2"]
)
corr = float(np.corrcoef(modes, shift)[0, 1])
print(f"corr(mode of the fitted density, the linear predictor) = {corr:.4f}")
assert corr > 0.95, f"the density's mode does not track the shifts: r = {corr:.4f}"

ok worst |integral of the density - 1|: 1.29e-03 <= 5.00e-03
corr(mode of the fitted density, the linear predictor) = 0.9951

```

Classical against Adam: the same optimum. A converged Adam fit with no early stopping reaches the same coefficients. The guarantees of the classical fit are determinism and the exact maximum-likelihood estimate, not speed. Which one is faster depends on the model. It wins clearly on ordinal-outcome models, as in Section 2, while on this Bernstein-heavy continuous DAG the flat directions of the basis slow L-BFGS down. Both timings are printed below rather than claimed, because a wall clock is a property of the machine.

```

flow_a = CausalFlowDAG(spec_vaca, seed=0)
t0 = time.perf_counter()
sched = PerNodeEarlyStopping(patience=60, lr_patience=15)
flow_a.fit(
    df,
    epochs=2000,
    batch_size=4096,
    validation_data=df,
    optimizer=per_node_adam(flow_a, lr=1e-1),
    callbacks=sched,
)
t_adam = time.perf_counter() - t0

```

```

coef_c, coef_a = flow_c.ls_coefficients(), flow_a.ls_coefficients()
print(f"{'coef':<10}{'classical':>12}{'adam':>12}{'|diff|':>10}")
for node, p in [("x2", "x1"), ("x3", "x1"), ("x3", "x2")]:
    c, aw = coef_cnode[0], coef_anode[0]
    print(f"{p}->{node:<6}{c:>12.4f}{aw:>12.4f}{abs(c - aw):>10.4f}")
print(f"\n\nclassical {rep['seconds']:.2f}s vs adam {t_adam:.1f}s")

coef      classical      adam      |diff|
x1->x2      1.7998      1.7944      0.0055
x1->x3     -1.7806     -1.7638      0.0168
x2->x3     -0.4550     -0.4461      0.0090

classical 3.65s vs adam 3.3s

```

2. Ordinal case

For an **ordinal** outcome the classical model is the ordered logit, which statsmodels.OrderedModel fits, so here the equivalence is checkable in Python. The data is the paper's mixed triangle (experiments/data/triangle-mixed/linear): x3 has four levels and the two continuous parents x1 and x2; docs/paper-replication.md states the DGP.

```

tm = pd.read_csv(DATA / "triangle-mixed" / "linear" / "obs.csv")

spec_tm = {
    "x1": ContinuousNode(),
    "x2": ContinuousNode(LS("x1")),
    "x3": OrdinalNode(4, LS("x1") + LS("x2")),
}

flow_o = CausalFlowDAG(spec_tm, seed=0)
flow_o.fit_classical(tm)

{'converged': False,
 'stop_reason': 'max_iter',
 'n_iter': 400,
 'final_nll': 1.0638607197240275,
 'grad_norm': 2.1249215878692695e-05,
 'seconds': 7.232787695999946,
 'coefficients': {'x2': {'x1': array([2.0299984])},
 'x3': {'x1': array([0.00946229]), 'x2': array([0.38318766])}}}

```

Refitting the same design in statsmodels

flow.design_matrix(node, drop_first=True) gives the encoding the flow feeds its own shifts, so handing it to statsmodels compares two fits of one design rather than two different designs.

```

design = flow_o.design_matrix(tm, "x3", drop_first=True)
res = OrderedModel(tm["x3"].astype(int), design, distr="logit").fit(
    method="bfgs", disp=False
)

fitted = flow_o.ls_coefficients()["x3"]
print(f"{'coefficient':<14}{'fit_classical':>14}{'statsmodels':>13}{'|diff|':>9}")
for name in ("x1", "x2"):
    a_, b_ = float(fittedname), res.params[name]
    print(f"{name:<14}{a_:>14.4f}{b_:>13.4f}{abs(a_ - b_):>9.4f}")

coefficient    fit_classical    statsmodels    |diff|

```

x1	0.0095	0.0097	0.0002
x2	0.3832	0.3832	0.0001

Standard errors

The helper from Section 1 needs no change: give it the node, and statsmodels reports the same quantities, so every number below has an external check.

```
table_o, diag_o = ls_conf_int(flow_o, "x3", tm)
ci_sm = res.conf_int()
print(
    f"{'':<6}{'flow SE':>9}{'sm SE':>8}{'|est|/SE':>10}    "
    f"{'flow 95%':>18}{'statsmodels 95%':>20}"
)
for term in ("x1", "x2"):
    r = table_o.loc[term]
    print(
        f"{term:<6}{r['se']:>9.4f}{res.bse[term]:>8.4f}"
        f"{abs(r['estimate']) / r['se']:>10.2f}    "
        f"[{r['lower']:+.3f}, {r['upper']:+.3f}]    "
        f"[{ci_sm.loc[term, 0]:+.3f}, {ci_sm.loc[term, 1]:+.3f}]"
    )

se_gap = max(
    abs(float(table_o.loc[term, "se"]) - float(res.bse[term])) for term in ("x1", "x2")
)
claim("max |SE_flow - SE_statsmodels|", se_gap, 1e-3)
```

	flow SE	sm SE	est /SE	flow 95%	statsmodels 95%
x1	0.1080	0.1080	0.09	[-0.202, +0.221]	[-0.202, +0.221]
x2	0.0735	0.0735	5.21	[+0.239, +0.527]	[+0.239, +0.527]

```
ok max |SE_flow - SE_statsmodels|: 1.23e-06 <= 1.00e-03
```

The standard errors agree with statsmodels. The |diff| column above is not evidence about identification: it is the optimizer stopping while the likelihood is still flat, and the cell below checks that the gap stays a small fraction of one standard error.

```
print(f"|grad| at the classical optimum: {diag_o['grad_norm']:.1e}")
for term in ("x1", "x2"):
    observed = abs(float(table_o.loc[term, "estimate"]) - float(res.params[term]))
    print(
        f"{term:<4} |flow - statsmodels| = {observed:.2e}"
        f"    = {observed / float(table_o.loc[term, 'se']):.0%} of one SE"
    )
    claim(f"{term}: gap against statsmodels", observed, 5e-2)
```

```
|grad| at the classical optimum: 4.2e-02
x1 |flow - statsmodels| = 1.95e-04 = 0% of one SE
ok x1: gap against statsmodels: 1.95e-04 <= 5.00e-02
x2 |flow - statsmodels| = 6.15e-05 = 0% of one SE
ok x2: gap against statsmodels: 6.15e-05 <= 5.00e-02
```

3. Warm-start handoff: classical fit → further training

fit_classical leaves the model at the MLE in float32, ready for any normal operation. Two things to verify:

1. the float64→float32 round-trip didn't move the coefficients, and

2. continuing with `fit()` from the classical solution *stays put* — confirming it really is the optimum (and showing the classical fit as a fast, principled initialization for further or richer training).

```

before = {k: v.copy() for k, v in flow_o.ls_coefficients()["x3"].items()}

# continue training from the classical solution with a gentle Adam phase
flow_o.fit(tm, epochs=300, learning_rate=1e-3, batch_size=256)
after = flow_o.ls_coefficients()["x3"]

print("coefficient drift after 300 more Adam epochs from the classical MLE:")
for p in ["x1", "x2"]:
    d = float(np.abs(after[p] - before[p]).max())
    print(f"  {p:<8} max|Δ| = {d:.4f}")
print("\n-> small drift = the classical fit was already at the optimum;")
print("  fit_classical is a valid warm start for continued / flexible training.")

coefficient drift after 300 more Adam epochs from the classical MLE:
x1      max|Δ| = 0.0027
x2      max|Δ| = 0.0013

-> small drift = the classical fit was already at the optimum;
  fit_classical is a valid warm start for continued / flexible training.

```

TRAM-DAG: one causal model, all three rungs

A TRAM-DAG (Sick & Dürr, CLear 2025) is one normalizing flow wired like the adjacency matrix of a causal DAG. Fit it once on observational data. Then answer all three rungs of Pearl's ladder with the same fitted model:

rung	query	call
L1 association	$p(x)$, sampling	<code>flow.log_prob(df), flow.sample(n)</code>
L2 intervention	$p(x \mid do(x_j=a))$	<code>flow.sample(n, do={...})</code>
L3 counterfactual	what would x_i have been, had x_j been a ?	<code>u = flow.abduct(df), then flow.sample(do={...}, u=u)</code>

This notebook uses the first benchmark of the paper. It is a three-variable process with a bimodal source, and its ground truth is known analytically. Every claim below is therefore a computed check, not a statement. A failed check stops the notebook.

The model theory is in `docs/model.md`. How to read a fitted model is in `docs/interpretation.md`.

The notebook runs on CPU in well under five minutes.

```

import importlib.util
import subprocess
import sys
import time

if importlib.util.find_spec("tramdag") is None: # Colab: install from PyPI
    subprocess.check_call(
        [sys.executable, "-m", "pip", "install", "-q", "tramdag[plots]"]
    )
# to track active development instead of the latest release, use:

```

```
# pip install "git+https://github.com/tensorchiefs/tramdag.git@main"

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch

from tramdag import CausalFlowDAG, ContinuousNode, I, plot_dag
from tramdag.callbacks import PerNodeEarlyStopping, per_node_adam
from tramdag.plots import plot_marginals, plot_training

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
plt.rcParams["figure.dpi"] = 110
t_total = time.perf_counter()
print(f"torch {torch.__version__} device: {DEVICE}")
torch 2.14.0+cu130 device: cpu
```

1. The benchmark process

The process comes from Sánchez-Martín et al. 2022, App. E.1, and from App. C.1 of the TRAM-DAG paper:

$$\begin{aligned}x_1 &\sim \tfrac{1}{2} \mathcal{N}(-2, 1.5) + \tfrac{1}{2} \mathcal{N}(1.5, 1) \quad (\text{bimodal source}) \\x_2 &= -x_1 + \mathcal{N}(0, 1) \\x_3 &= x_1 + 0.25 x_2 + \mathcal{N}(0, 1)\end{aligned}$$

The DAG is $x_1 \rightarrow x_2$, $x_1 \rightarrow x_3$, $x_2 \rightarrow x_3$. The source is bimodal, which is the part that a location-scale model cannot fit; the monotone transform of each node is what fits it (docs/model.md).

```
# The process is written out here, so the notebook needs nothing but tramdag.
# draw_latents and simulate stay separate on purpose. Keeping the noise lets
# section 5 intervene on the same individuals, which is what makes an
# individual counterfactual checkable.
```

```
def draw_latents(n, rng):
    """Draw every noise variable of the process, n rows each."""
    return {
        "x1_mix": rng.uniform(size=n), # which mixture component
        "x1_a": rng.normal(size=n), # the N(-2, 1.5) branch
        "x1_b": rng.normal(size=n), # the N(1.5, 1) branch
        "x2": rng.normal(size=n),
        "x3": rng.normal(size=n),
    }

def simulate(latents, do=None):
    """Run the process forward. A variable named in `do` is clamped instead."""
    do = do or {}
    n = len(latents["x2"])

    if "x1" in do:
        x1 = np.full(n, float(do["x1"]))
    else:
```

```

x1 = np.where(
    latents["x1_mix"] < 0.5,
    -2.0 + np.sqrt(1.5) * latents["x1_a"],
    1.5 + latents["x1_b"],
)

x2 = np.full(n, float(do["x2"])) if "x2" in do else -x1 + latents["x2"]
x3 = x1 + 0.25 * x2 + latents["x3"]
return pd.DataFrame({"x1": x1, "x2": x2, "x3": x3})

def sample_dgp(n, seed, do=None):
    """Draw n fresh rows, optionally under an intervention."""
    return simulate(draw_latents(n, np.random.default_rng(seed)), do)

N = 20_000
df = sample_dgp(N, seed=43)
train, val = df.iloc[: N - 2_000], df.iloc[N - 2_000 :]

fig, ax = plt.subplots(figsize=(5.5, 3))
ax.hist(df["x1"], bins=80, density=True, alpha=0.7)
ax.set_title(f"the source  $x_1$  has two modes ({N:,} rows)")
ax.set_xlabel(" $x_1$ ")
fig.tight_layout()
plt.show()

```

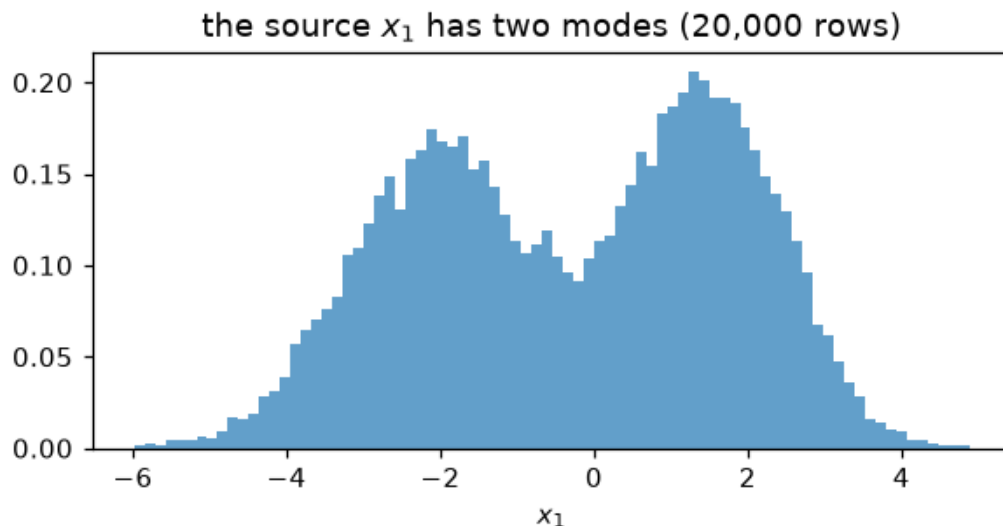


Figure 2: png

2. The spec is the DAG, and one fit

A spec is one entry per node. Each entry names the terms through which the parents enter. `I(...)` puts the parents in control of the transform parameters, which is the most flexible term. `notebooks/classical_fit_tram_dag.py` writes the same DAG with interpretable terms.

One Adam fits every node at once (`docs/fitting.md`). `validation_data=` scores the validation rows per epoch, `verbose=` prints progress, and the strategy attaches through `callbacks=`:

per_node_adam with PerNodeEarlyStopping freezes each node once its own validation score has plateaued and loads its best weights back, so epochs=200 is a ceiling and not a budget.

notebooks/training_strategies.py works through the other recipes.

```
spec = {
    "x1": ContinuousNode(), # the source has no parents
    "x2": ContinuousNode(I("x1")),
    "x3": ContinuousNode(I("x1", "x2")),
}
plot_dag(spec)
plt.show()
```

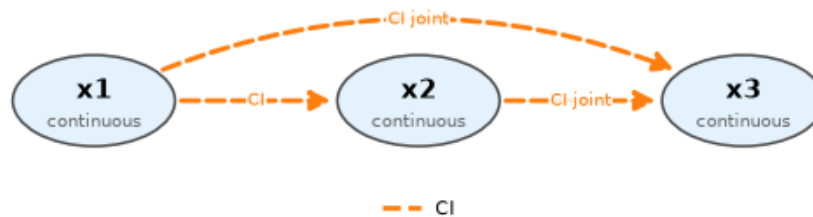


Figure 3: png

```
torch.manual_seed(0)
flow = CausalFlowDAG(spec, device=DEVICE)
sched = PerNodeEarlyStopping(patience=40, lr_patience=10)

t0 = time.perf_counter()
flow.fit(
    train,
    epochs=200,
    batch_size=2048,
    validation_data=val,
    verbose=50,
    optimizer=per_node_adam(flow, lr=1e-1),
    callbacks=sched,
)
t_fit = time.perf_counter() - t0
epochs_used = len(flow.history["val"])
print(f"\nfitted on {DEVICE} in {t_fit:.1f}s, {epochs_used} epochs")
print(f"each node froze at epoch: {dict(sorted(sched.frozen.items()))}")

# The ceiling must not bind. If it does, the plateau rule never finished and
# the numbers below describe an unconverged fit.
assert epochs_used < 200, (
    "the fit used all 200 epochs, so no node froze and the plateau rule did "
    "not self-stop; raise the ceiling before trusting anything below"
)

epoch 50/200  train 4.8922  val 4.9353

epoch 83/200  train 4.8954  val 4.9340
```

```
fitted on cpu in 10.4s, 83 epochs
each node froze at epoch: {'x1': 63, 'x2': 83, 'x3': 55}
plot_training(flow, frozen=sched.frozen)
plt.show()
```

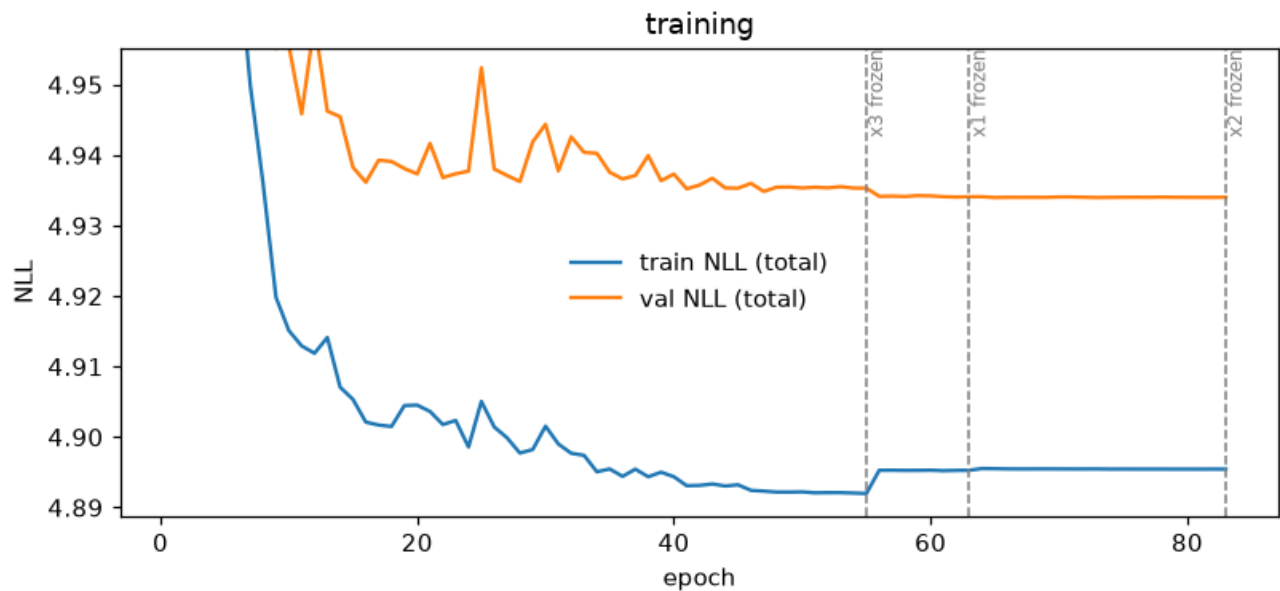


Figure 4: png

3. Rung 1: the observational fit

Samples from the fitted flow must reproduce the observed joint distribution. The marginals are the picture. The correlation matrix is the check, because it also covers the dependence between variables, which a marginal plot does not show.

```
plot_marginals(flow, df, seed=1)
plt.show()
```

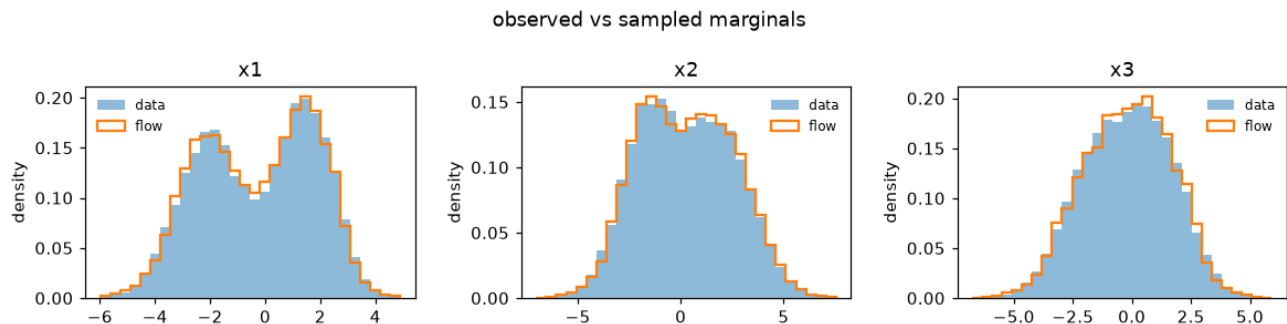


Figure 5: png

```
samp = flow.sample(len(df), seed=1)
corr_gap = float(np.abs(samp.corr().to_numpy() - df.corr().to_numpy()).max())
print(f"largest absolute difference between correlation matrices: {corr_gap:.4f}")
assert corr_gap < 0.05, f"the sampled dependence structure is off by {corr_gap:.4f}"
largest absolute difference between correlation matrices: 0.0172
```

4. Rung 2: interventions

do= mutilates the graph. It clamps x_2 , cuts the edge into it, and resamples everything downstream. Under the process, $x_3 \mid do(x_2=a) = x_1 + 0.25a + \mathcal{N}(0,1)$, so $\mathbb{E}[x_3] = -0.25 + 0.25a$ exactly. That analytic target is what makes an error visible.

```
fig, axes = plt.subplots(1, 3, figsize=(11, 3.2), sharey=True)
errors = []
print("E[x3 | do(x2=a)]      analytic      TRAM-DAG      error")
for ax, a in zip(axes, (-3.0, -1.0, 0.0), strict=True):
    truth = sample_dgp(len(df), seed=543, do={"x2": a})
    fitted = flow.sample(len(df), do={"x2": a}, seed=2)
    bins = np.linspace(truth["x3"].quantile(0.001), truth["x3"].quantile(0.999), 70)
    ax.hist(truth["x3"], bins=bins, density=True, alpha=0.5, label="process")
    ax.hist(
        fitted["x3"], bins=bins, density=True, histtype="step", lw=1.6, label="TRAM-DAG"
    )
    ax.set_title(f"$p(x_3 \mid do(x_2={a:+.0f}))$")

    analytic = -0.25 + 0.25 * a
    got = float(fitted["x3"].mean())
    errors.append(abs(got - analytic))
    print(
        f"      a = {a:+.0f}:           {analytic:+.4f}           {got:+.4f}           {errors[-1]:.4f}"
    )
axes[0].legend()
fig.tight_layout()
plt.show()

# The bound is about three times the largest error measured while writing this
# notebook (0.048), which leaves room for another machine and another draw.
assert max(errors) < 0.15, f"interventional mean off by {max(errors):.4f}"

E[x3 | do(x2=a)]      analytic      TRAM-DAG      error
```

a = -3:	-1.0000	-1.0532	0.0532
a = -1:	-0.5000	-0.5338	0.0338
a = +0:	-0.2500	-0.2997	0.0497

5. Rung 3: individual counterfactuals

Take 1,000 held-out individuals. Abduction inverts the flow and recovers the latent noise of each one. That noise is everything about the individual that the model does not attribute to the parents. Pushing it back through the flow must return the observed row, which is the first check.

Action and prediction then rerun each individual under $do(x_1 = 0)$ with the same noise. The process keeps its own noise, so the true individual counterfactual is known and the flow can be scored per individual. Real data never permits this check, because only one of the two outcomes exists.

```
lat = draw_latents(1_000, np.random.default_rng(7))
factual = simulate(lat) # the same noise on both sides,
cf_true = simulate(lat, do={"x1": 0.0}) # so these are true counterfactuals
```

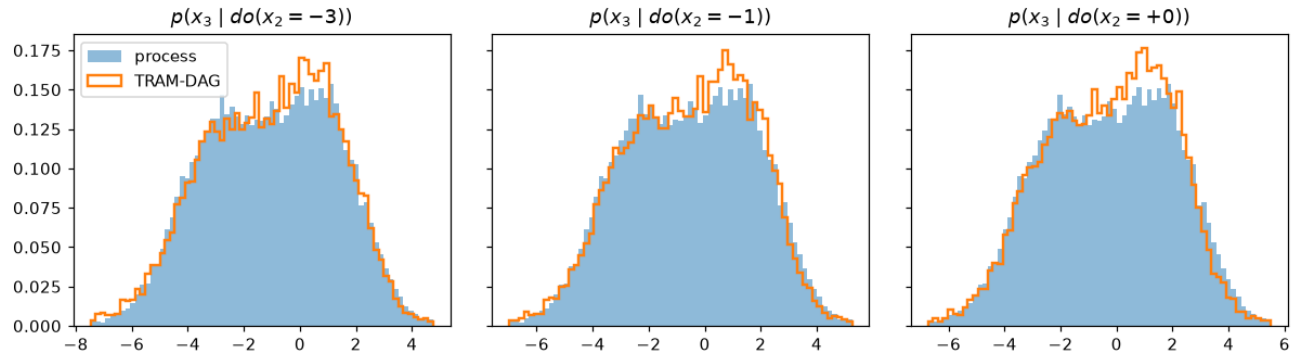


Figure 6: png

```

u = flow.abduct(factual)
recon_err = float(np.abs(flow.sample(u=u).to_numpy() - factual.to_numpy()).max())
print(f"abduction then reconstruction: max |error| = {recon_err:.2e}")
# This inverts a monotone transform in float32, so the bound is a precision
# bound rather than a statistical one.
assert recon_err < 1e-4, f"abduction did not round-trip: {recon_err:.2e}"

cf_flow = flow.sample(do={"x1": 0.0}, u=u)
fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
for ax, c in zip(axes, ["x2", "x3"], strict=True):
    r = float(np.corrcoef(cf_true[c], cf_flow[c])[0, 1])
    ax.scatter(cf_true[c], cf_flow[c], s=4, alpha=0.4)
    lims = [cf_true[c].min(), cf_true[c].max()]
    ax.plot(lims, lims, "k--", lw=1)
    ax.set_title(f"${c[0]}_{c[1]}$ under $do(x_1{{=}}0)$ (r = {r:.4f})")
    ax.set_xlabel("true, shared noise")
    ax.set_ylabel("TRAM-DAG")
    assert r > 0.99, f"counterfactual {c} correlates only {r:.4f} with the truth"
fig.suptitle("L3: individual counterfactuals, scored one unit at a time")
fig.tight_layout()
plt.show()

abduction then reconstruction: max |error| = 5.50e-06

```

What this notebook checked

rung	call	checked against
L1	flow.sample(n)	the correlation matrix of the data
L2	flow.sample(n, do=...)	the analytic mean $-0.25 + 0.25a$
L3	flow.abduct(df), then flow.sample(do=..., u=u)	the true per-unit counterfactual

Where to go next:

- docs/model.md, the model and its notation.
- docs/interpretation.md, reading a fitted model, including ordinal nodes and log-odds ratios.

L3: individual counterfactuals, scored one unit at a time

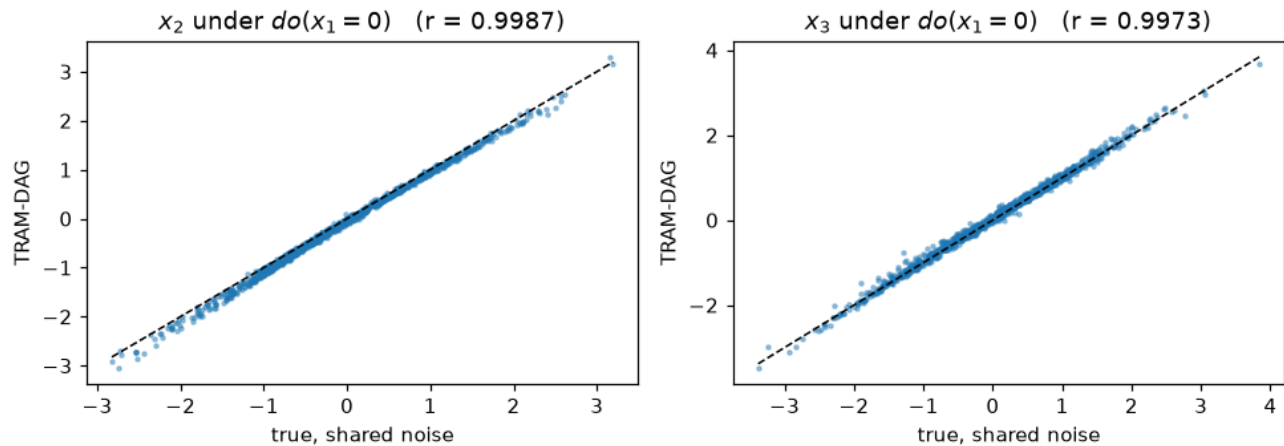


Figure 7: png

- notebooks/training_strategies.py, the fitting API and every shipped strategy.
- notebooks/classical_fit_tram_dag.py, the agreement with statsmodels and R.
- notebooks/varying_coefficients.py, effects that vary with a covariate.
- repo · paper

```
elapsed = time.perf_counter() - t_total
print(f"whole notebook: {elapsed:.0f}s")
assert elapsed < 300, f"the notebook took {elapsed:.0f}s, over its five-minute claim"
whole notebook: 13s
```

Training strategies: driving fit from the API side

fit is one minibatch Adam loop that keeps the final weights. It owns three things only: the loop, the per-epoch validation score, and progress printing. Everything else is the caller's, through optimizer= and callbacks=.

This notebook runs every shipped strategy on one small workload, so the comparison is like for like, and prints what each one leaves behind in flow.history, and closes with a runtime comparison of the recipes on this one workload. For which strategy to pick see the table in docs/fitting.md.

```
import time
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch

from tramdag import CausalFlowDAG, ContinuousNode, I
from tramdag.callbacks import (
    Callback,
    EarlyStopping,
    PerNodeEarlyStopping,
    per_node_adam,
```

```

)
from tramdag.plots import plot_training

plt.rcParams["figure.dpi"] = 110

# repo-relative data, whether the notebook runs from the repo root or notebooks/
REPO = next(
    p for p in [Path.cwd(), *Path.cwd().parents] if (p / "pyproject.toml").exists()
)

# The tracked 1000-row VACA sample, the same benchmark the Colab demo uses.
df = pd.read_csv(REPO / "notebooks" / "data" / "vaca.csv")
train, val = df.iloc[:900], df.iloc[900:]

SPEC = {
    "x1": ContinuousNode(),
    "x2": ContinuousNode(I("x1")),
    "x3": ContinuousNode(I("x1", "x2")),
}
CEILING = 300 # epoch ceiling for every recipe below
scoreboard = []

def build():
    """Give a fresh unfitted flow, seeded so every recipe starts identically."""
    torch.manual_seed(0)
    return CausalFlowDAG(SPEC)

def record(name, flow, seconds):
    """Add one finished recipe to the scoreboard and print its line."""
    nll = sum(flow.nll(val).values())
    epochs = len(flow.history["train"])
    scoreboard.append(
        {"strategy": name, "val_nll": nll, "epochs": epochs, "seconds": seconds}
    )
    print(f"{name:22s} val NLL {nll:.4f} {epochs:4d} epochs {seconds:5.1f}s")
    return nll

print(f"{len(train)} train rows, {len(val)} validation rows")
900 train rows, 100 validation rows

```

1. What fit records without any callback

Pass `validation_data=` and fit scores the validation set once per epoch, centrally. Every shipped callback reads that one computation rather than repeating it. Three keys appear in `flow.history`:

- `train`, one dict of per-node negative log-likelihood per epoch,
- `val`, the same for the validation rows, only when validation is on,
- `lr`, the optimizer's rate after each epoch, recorded after the callbacks ran, so a schedule's decision for that epoch is what gets stored.

```

flow = build()
flow.fit(train, epochs=20, batch_size=256, validation_data=val)

```

```

print("history keys:", sorted(flow.history))
print("epochs recorded:", len(flow.history["train"]))
print("per-node train NLL, last epoch:")
for node, value in flow.history["train"].items():
    print(f"    {node}: {value:.4f}")
print("learning rate, last epoch:", flow.history["lr"])

assert len(flow.history["train"]) == len(flow.history["val"]) == 20
assert set(flow.history["train"]) == set(SPEC)

history keys: ['lr', 'train', 'val', 'val_epoch']
epochs recorded: 20
per-node train NLL, last epoch:
    x1: 2.6743
    x2: 1.4033
    x3: 1.4267
learning rate, last epoch: {0: 0.01}

```

Without `validation_data=` there is no `val` key, and the shipped callbacks refuse rather than guess.

```

bare = build()
bare.fit(train, epochs=5, batch_size=256)
print("history keys without validation:", sorted(bare.history))
assert "val" not in bare.history

try:
    build().fit(train, epochs=5, batch_size=256, callbacks=EarlyStopping())
except RuntimeError as err:
    print("\nEarlyStopping without validation refuses:\n    ", err)
else:
    raise AssertionError("EarlyStopping should refuse without validation data")

history keys without validation: ['lr', 'train']

EarlyStopping without validation refuses:
    EarlyStopping reads flow.history['val'] – pass validation_data= or validation_split= to fit()

```

2. Plain Adam

One rate, one loop, and the weights at the last epoch. This is the baseline every other recipe is measured against. An all-LS model trained this way to convergence reaches the classical maximum-likelihood estimate, which `classical_fit_tram_dag.py` checks against `statsmodels` and `R`.

```

flow = build()
t0 = time.perf_counter()
flow.fit(train, epochs=CEILING, batch_size=256, learning_rate=1e-2, validation_data=val)
nll_plain = record("plain Adam", flow, time.perf_counter() - t0)
history_plain = [sum(d.values()) for d in flow.history["val"]]

plain Adam          val NLL 4.9958    300 epochs    6.4s

```

3. Two phases

Calling `fit` again continues training. The optimizer is rebuilt unless you pass your own, so a second call at a lower rate is a coarse-then-fine schedule with no callback at all. History accumulates across the calls.

```

flow = build()
t0 = time.perf_counter()
flow.fit(train, epochs=200, batch_size=256, learning_rate=1e-2, validation_data=val)
after_coarse = sum(flow.nll(val).values())
flow.fit(train, epochs=100, batch_size=256, learning_rate=1e-3, validation_data=val)
nll_two_phase = record("two phases", flow, time.perf_counter() - t0)

print(f"    after the coarse phase: {after_coarse:.4f}")
print(f"    after the fine phase:    {nll_two_phase:.4f}")
assert len(flow.history["train"]) == 300, "the second fit did not continue the history"

two phases          val NLL 4.9781    300 epochs    6.1s
    after the coarse phase: 5.0057
    after the fine phase:   4.9781

```

4. EarlyStopping: keep the best weights

The loop keeps the final weights, which is what makes the classical agreement exact. A flexible model does not always want that: it can reach a better validation score part way through and then drift. EarlyStopping snapshots the best epoch and loads it back at the end of the fit.

The check below is the one a static code block cannot make. After the fit, the model's validation score equals the *minimum* over the recorded epochs, not the last one.

```

flow = build()
t0 = time.perf_counter()
stopper = EarlyStopping() # restore_best is on by default, no patience
flow.fit(
    train,
    epochs=CEILING,
    batch_size=256,
    learning_rate=1e-2,
    validation_data=val,
    callbacks=stopper,
)
nll_best = record("EarlyStopping", flow, time.perf_counter() - t0)

recorded = [sum(d.values()) for d in flow.history["val"]]
print(f"    best epoch {stopper.best_epoch} of {len(recorded)}")
print(f"    minimum recorded: {min(recorded):.4f}")
print(f"    last epoch:        {recorded[-1]:.4f}")
assert abs(nll_best - min(recorded)) < 1e-4, (
    "the restored weights do not match the best recorded epoch"
)

EarlyStopping      val NLL 4.9564    300 epochs    6.0s
    best epoch 180 of 300
    minimum recorded: 4.9564
    last epoch:      4.9958

```

5. EarlyStopping(patience=): stop as well as restore

Without patience the fit spends its whole budget and only the restoration happens. With it, the fit also stops once the best epoch is that many epochs old.

```

flow = build()
t0 = time.perf_counter()
stopper = EarlyStopping(patience=25)

```

```

flow.fit(
    train,
    epochs=CEILING,
    batch_size=256,
    learning_rate=1e-2,
    validation_data=val,
    callbacks=stopper,
)
record("EarlyStopping(25)", flow, time.perf_counter() - t0)

spent = len(flow.history["train"])
print(f"    stopped after {spent} of {CEILING} epochs, best was {stopper.best_epoch}")
assert spent < CEILING, "patience never triggered, so the ceiling bound instead"
assert spent - stopper.best_epoch >= 25
EarlyStopping(25)      val NLL 4.9564    205 epochs    4.2s
    stopped after 205 of 300 epochs, best was 180

```

6. Per-node rates: per_node_adam with PerNodeEarlyStopping

per_node_adam builds an Adam with one tagged parameter group per node, and PerNodeEarlyStopping decays, freezes and restores each node on its own validation score (docs/fitting.md). The fit stops when the last node freezes.

Do not attach a torch scheduler to the same optimizer. Two controllers would steer the same group rates against each other.

```

flow = build()
t0 = time.perf_counter()
plateau = PerNodeEarlyStopping(patience=40, lr_patience=10)
flow.fit(
    train,
    epochs=CEILING,
    batch_size=256,
    validation_data=val,
    optimizer=per_node_adam(flow, lr=1e-2),
    callbacks=plateau,
)
record("PerNodeEarlyStopping", flow, time.perf_counter() - t0)

print(f"    froze at: {dict(sorted(plateau.frozen.items()))}")
print("    per-node rates at the end:", flow.history["lr"])
assert set(plateau.frozen) == set(SPEC), "not every node froze"
assert len(flow.history["train"]) < CEILING, "the plateau rule did not self-stop"
PerNodeEarlyStopping  val NLL 4.9852    237 epochs    5.3s
    froze at: {'x1': 237, 'x2': 85, 'x3': 58}
    per-node rates at the end: {'x1': 0.0, 'x2': 0.0, 'x3': 0.0}

history["lr"] holds one dict per epoch, keyed by the node tags here, so the decay of each node
is on record without a callback of your own. plot_training marks the freeze epochs.

plot_training(flow, frozen=plateau.frozen)
plt.show()

```

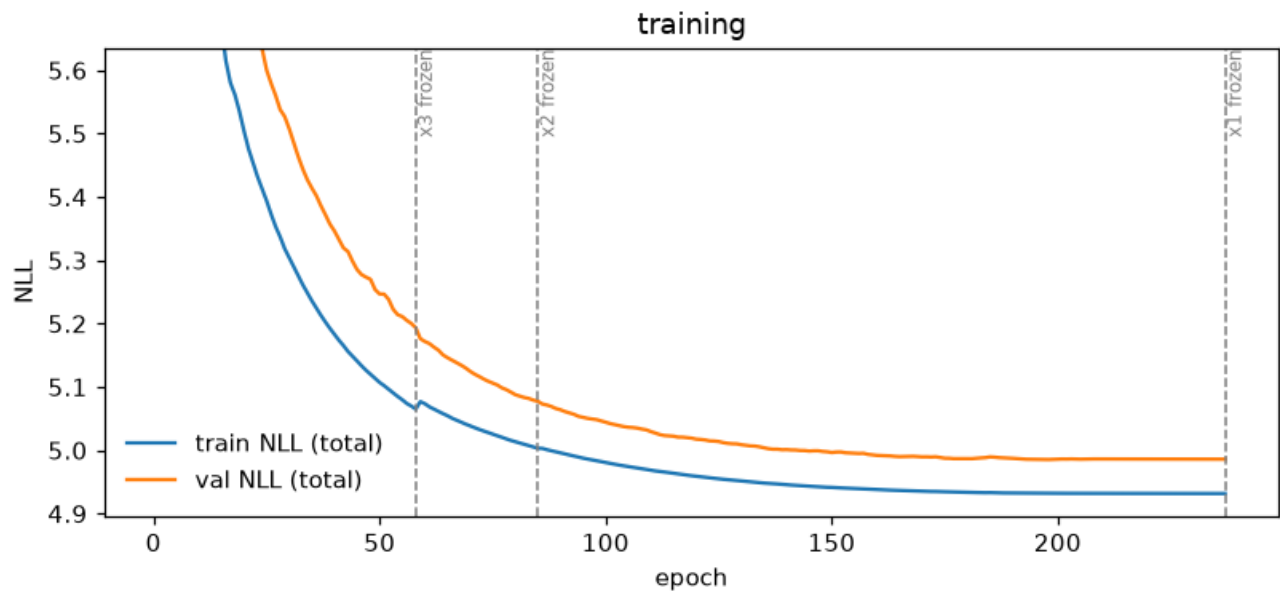


Figure 8: png

7. Writing your own

The callback contract is in docs/fitting.md: a bare callable is an `on_epoch_end` hook, and a `Callback` subclass gets the other two hooks. `GlobalPlateau` below carries torch's `ReduceLROnPlateau` on the validation score fit has already put in `history["val"]`.

```
class GlobalPlateau(Callback):
    """One torch ReduceLROnPlateau over all nodes, driven by fit's own score."""

    def __init__(self, factor=0.3, patience=10):
        self.factor, self.patience = factor, patience
        self.scheduler = None

    def on_fit_begin(self, flow, optimizer):
        """Build the scheduler here: the optimizer exists only once fit runs."""
        self.scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
            optimizer, factor=self.factor, patience=self.patience
        )

    def on_epoch_end(self, self, flow, epoch, optimizer):
        """Step the scheduler, and stop once the rate has bottomed out."""
        self.scheduler.step(sum(flow.history["val"].values()))
        return optimizer.param_groups[0] < 1e-5
```

```
flow = build()
optimizer = torch.optim.Adam(flow.parameters(), lr=1e-2)
t0 = time.perf_counter()
flow.fit(
    train,
    epochs=CEILING,
    batch_size=256,
    validation_data=val,
```

```

optimizer=optimizer,
callbacks=GlobalPlateau(),
)
record("GlobalPlateau", flow, time.perf_counter() - t0)

final_lr = optimizer.param_groups0
print(f"    rate went 1.0e-02 -> {final_lr:.1e}")
assert final_lr < 1e-2, "the scheduler never decayed the rate"

GlobalPlateau      val NLL 4.9888    210 epochs    4.3s
    rate went 1.0e-02 -> 7.3e-06

```

A one-line callable is often enough. This one records a coefficient after every epoch, which is how the paper replications trace convergence:

```

trace = []
flow.fit(train, epochs=100, validation_data=val,
         callbacks=lambda f, epoch, opt: trace.append(f.nll(val)["x3"]))

```

A callable of the wrong shape is refused before the first epoch, so a mis-registered callback cannot waste a long run.

```

try:
    build().fit(train, epochs=5, batch_size=256, callbacks=lambda flow: None)
except TypeError as err:
    print("wrong arity refused up front:\n    ", err)
else:
    raise AssertionError("a one-argument callable should be refused")

```

```

wrong arity refused up front:
<lambda>() takes 1 positional argument but 3 were given

```

8. The scoreboard: a poor man's runtime comparison

One workload, one seed, one machine, every recipe: the wall-clock seconds each one took, the epochs it spent, its validation NLL and the gap to the best NLL on the board. A recipe that stops itself wins on seconds only if it also stays near the best NLL, which is what the last two columns show side by side. Absolute seconds are this machine's; the ranking is what travels.

```

board = pd.DataFrame(scoreboard).set_index("strategy")
board["nll_gap"] = board["val_nll"] - board["val_nll"].min()
board["s_per_epoch"] = board["seconds"] / board["epochs"]
print(board.to_string(float_format=lambda v: f"{v:.4f}"))

```

```

fig, ax = plt.subplots(figsize=(7, 3.4))
ax.scatter(board["seconds"], board["nll_gap"])
for name, row in board.iterrows():
    ax.annotate(
        name,
        (row["seconds"], row["nll_gap"]),
        xytext=(4, 4),
        textcoords="offset points",
        fontsize=8,
    )
ax.set_xlabel("wall-clock seconds")
ax.set_ylabel("validation NLL above the best recipe")
ax.set_title("cost against quality, one workload")
fig.tight_layout()
plt.show()

```

```
# Keeping the best weights cannot score worse than keeping the last ones.
assert nll_best <= nll_plain + 1e-6, (
    f"EarlyStopping scored {nll_best:.4f} against plain Adam's {nll_plain:.4f}"
)
```

	val_nll	epochs	seconds	nll_gap	s_per_epoch
strategy					
plain Adam	4.9958	300	6.3732	0.0394	0.0212
two phases	4.9781	300	6.0854	0.0217	0.0203
EarlyStopping	4.9564	300	6.0361	0.0000	0.0201
EarlyStopping(25)	4.9564	205	4.1756	0.0000	0.0204
PerNodeEarlyStopping	4.9852	237	5.2593	0.0288	0.0222
GlobalPlateau	4.9888	210	4.2774	0.0324	0.0204

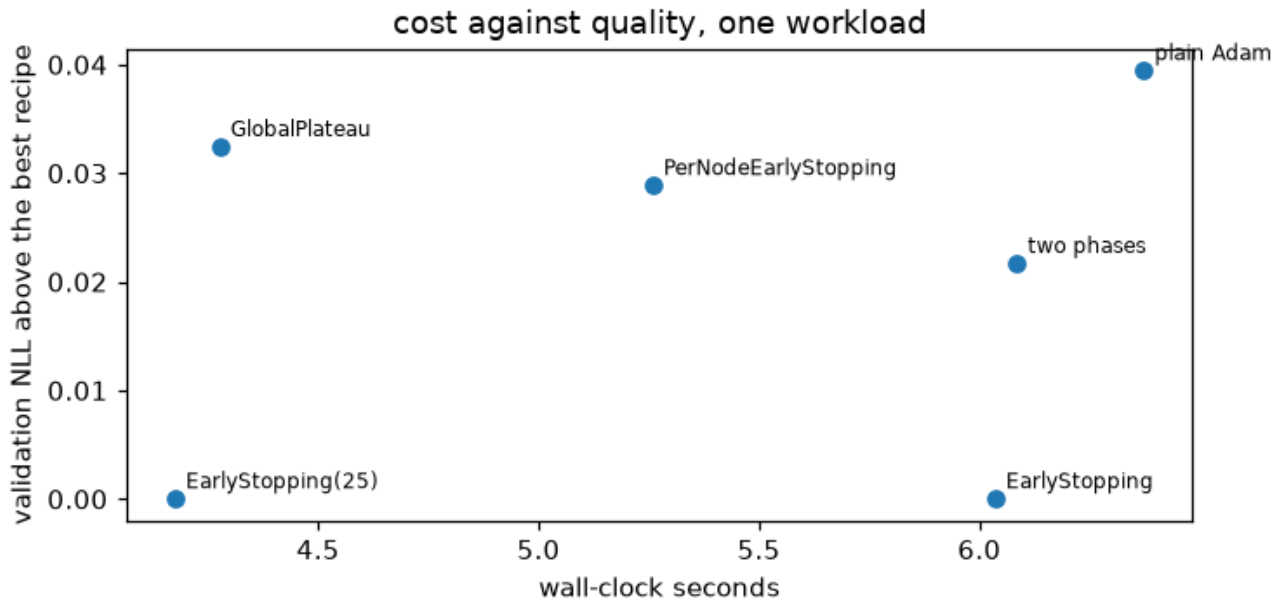


Figure 9: png

```
fig, ax = plt.subplots(figsize=(7, 3.4))
ax.plot(np.arange(1, len(history_plain) + 1), history_plain, label="plain Adam")
ax.axhline(nll_best, ls="--", lw=1, color="C1", label="EarlyStopping, restored")
ax.set_xlabel("epoch")
ax.set_ylabel("validation NLL (total)")
ax.set_ylim(min(history_plain) - 0.02, min(history_plain) + 0.4)
ax.legend()
ax.set_title("keeping the best epoch against keeping the last")
fig.tight_layout()
plt.show()
```

Which one, when

The decision table lives in docs/fitting.md.

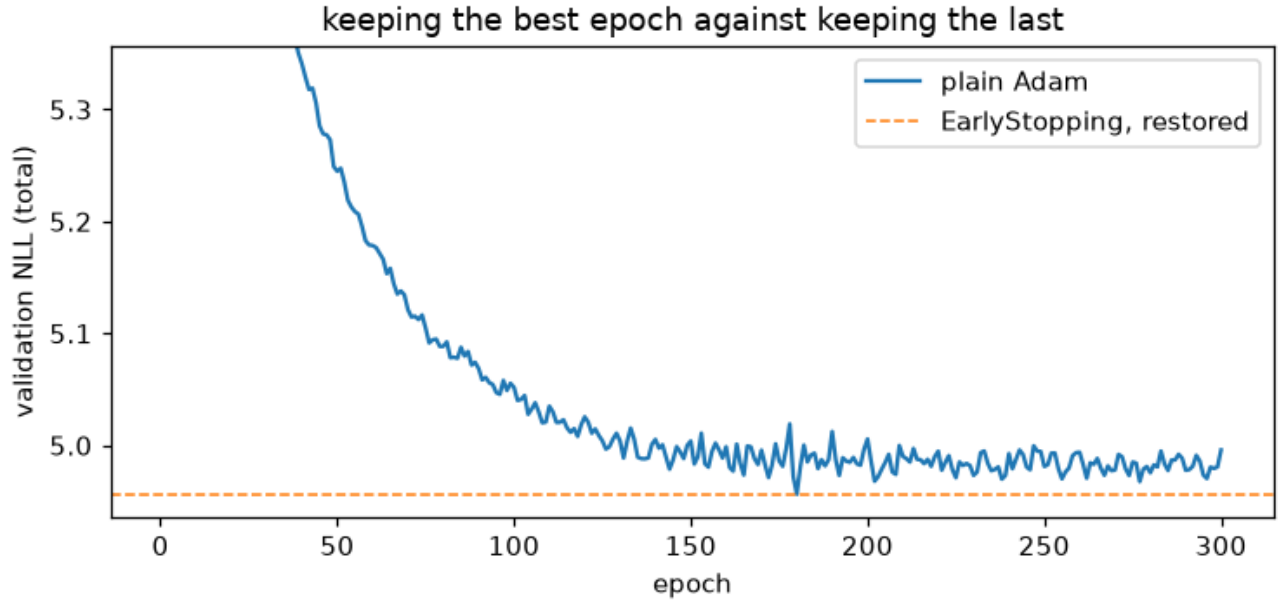


Figure 10: png

Heterogeneous treatment effects: the VC term

Most causal questions are not “does the treatment help?” but “**whom** does it help, and by how much?”. A TRAM-DAG answers that with a **varying-coefficient** term, which gives one edge of the DAG a treatment effect that depends on covariates:

$$\text{shift} += \beta(\text{modifiers}) \cdot x_t, \quad \beta(x) = \beta_0 + b_{\Theta}(x)$$

docs/varying-coefficients.md states the model, its penalty and its centering; this notebook runs it.

This notebook builds a DGP whose effect function we know exactly, fits the model, and scores the recovered $\beta(x)$ against the truth. It then shows the two things the term exists for that a plain flexible shift does not give: a read-out you can plot, and a **propensity-centered** variant that survives confounding.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from tramdag import CS, LS, VC, CausalFlowDAG, ContinuousNode, I, OrdinalNode
from tramdag.callbacks import EarlyStopping

plt.rcParams["figure.dpi"] = 110
```

1. A DGP with a known effect function

Three covariates, a **confounded** treatment (X1 and X2 push assignment), and an outcome whose conditional is exactly a logistic-latent flow:

$$h(y) + g(x) + \beta(x)t = U, \quad U \sim \text{Logistic}(0, 1)$$

with $h(y) = 2y$, a nonlinear prognostic part $g(x) = \frac{1}{2}X_1^2 + X_2 - \frac{1}{2}X_3$, and the effect function

$$\beta(x) = -1 + 0.8 X_2 - 0.6 X_3.$$

X_2 is deliberately *both* a confounder and an effect modifier — the case where a naive estimate fails hardest.

$B_0, B_2, B_3 = -1.0, 0.8, -0.6$

```
def true_beta(df):
    """The effect function the model has to recover, on the latent scale."""
    return B0 + B2 * df["X2"].to_numpy() + B3 * df["X3"].to_numpy()

def simulate(n, seed):
    rng = np.random.default_rng(seed)
    x1, x2, x3 = (rng.normal(size=n) for _ in range(3))
    t = (rng.logistic(size=n) > -(0.4 * x1 + 0.4 * x2)).astype(float)
    g = 0.5 * x1**2 + x2 - 0.5 * x3
    beta = B0 + B2 * x2 + B3 * x3 # the same expression true_beta reports
    y = (rng.logistic(size=n) - g - beta * t) / 2.0
    return pd.DataFrame({"X1": x1, "X2": x2, "X3": x3, "T": t, "Y": y})

train, val, test = simulate(4500, 1), simulate(500, 2), simulate(2000, 3)
print(f"treated fraction: {train['T'].mean():.2f}")
print(
    f"true effect ranges from {true_beta(test).min():+.2f} to {true_beta(test).max():+.2f}"
)

treated fraction: 0.48
true effect ranges from -4.41 to +2.29
```

2. Which covariates modify the effect? (effect_modifier_scan)

Before committing to a set of modifiers, ask the data. Fit the **cheap all-ls model** — seconds, deterministic — and look at the per-observation scores of the treatment coefficient. If the effect really were constant, those scores would carry no structure in any covariate; a CUSUM statistic per covariate turns that into a ranking.

```
def cheap_all_ls():
    return {
        "X1": ContinuousNode(),
        "X2": ContinuousNode(),
        "X3": ContinuousNode(),
        "T": OrdinalNode(2, LS("X1") + LS("X2")),
        "Y": ContinuousNode(LS("X1") + LS("X2") + LS("X3") + LS("T")),
    }
```

```
screen = CausalFlowDAG(cheap_all_ls(), seed=0)
screen.fit_classical(train)
print(screen.effect_modifier_scan(train, "Y", t="T"))
```

	stat	p_value	crit_5pct	flag
X2	4.163334	1.759830e-15	1.3581	True

X1	4.139814	2.600764e-15	1.3581	True
X3	3.162057	4.133820e-09	1.3581	True

X2 and X3 are the true modifiers and both flag — but so does X1, which does **not** enter $\beta(x)$ at all. Here g contains $\frac{1}{2}X_1^2$, which the cheap linear model cannot represent, and docs/scores.md explains why a misspecified prognostic part flags too.

The demonstration: re-generate with a *linear* prognostic X_1 , change nothing else, and X1 drops out of the shortlist.

```
def simulate_linear_x1(n, seed):
    rng = np.random.default_rng(seed)
    x1, x2, x3 = (rng.normal(size=n) for _ in range(3))
    t = (rng.logistic(size=n) > -(0.4 * x1 + 0.4 * x2)).astype(float)
    g = 0.5 * x1 + x2 - 0.5 * x3 # linear, so the cheap model fits it exactly
    y = (rng.logistic(size=n) - g - (B0 + B2 * x2 + B3 * x3) * t) / 2.0
    return pd.DataFrame({"X1": x1, "X2": x2, "X3": x3, "T": t, "Y": y})
```

```
well_specified = simulate_linear_x1(4500, 1)
screen2 = CausalFlowDAG(cheap_all_ls(), seed=0)
screen2.fit_classical(well_specified)
print(screen2.effect_modifier_scan(well_specified, "Y", t="T"))
```

	stat	p_value	crit_5pct	flag
X2	4.609058	7.066926e-19	1.3581	True
X3	3.476365	6.368441e-11	1.3581	True
X1	0.552602	9.201793e-01	1.3581	False

3. The spec: prognostic part and effect head are separate

CS("X1", "X2", "X3") absorbs the prognostic signal — as flexible as you like — while VC("X2", "X3", t="T") carries the effect. X2 and X3 appear twice on purpose: prognostically through the shift, and as modifiers through the head. Only the treatment named by t= owns its edge.

```
spec = {
    "X1": ContinuousNode(),
    "X2": ContinuousNode(),
    "X3": ContinuousNode(),
    "T": OrdinalNode(2, LS("X1") + LS("X2")),
    "Y": ContinuousNode(CS("X1", "X2", "X3") + VC("X2", "X3", t="T")),
}
flow = CausalFlowDAG(spec, seed=0)
flow.fit(
    train,
    epochs=300,
    learning_rate=1e-2,
    batch_size=512,
    validation_data=val,
    seed=0,
    callbacks=EarlyStopping(), # keep the best-validation weights
)
print(flow.to_matrix())
```

	X1	X2	X3	T	Y
X1				LS	CS['X1', 'X2', 'X3']
X2				LS	CS['X1', 'X2', 'X3']+VCm
X3					CS['X1', 'X2', 'X3']+VCm

T
Y

VC

In the meta-adjacency view (docs/interpretation.md) VC marks the treatment edge and VCm the modifiers, so X2 visibly enters Y twice.

4. Reading the effect out

varying_coef gives $\beta(x)$ per row. It is **deterministic** and does not look at the outcome — it is a property of the fitted model, not of the rows' Y values.

```
beta_hat = flow.varying_coef(test, "Y")
beta_true = true_beta(test)
corr = float(np.corrcoef(beta_hat, beta_true)[0, 1])
beta0 = float(flow.nodes["Y"].shifts["T"].beta0)

print(f"corr(beta_hat, beta_true) = {corr:.3f}")
print(f"beta0 = {beta0:+.3f} (true constant part {B0:+.1f})")
print(f"mean |error| = {np.abs(beta_hat - beta_true).mean():.3f}")

fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
axes[0].scatter(beta_true, beta_hat, s=6, alpha=0.35)
lims = [min(beta_true.min(), beta_hat.min()), max(beta_true.max(), beta_hat.max())]
axes[0].plot(lims, lims, "k--", lw=1)
axes[0].set_xlabel(r"true  $\beta(x)$ ")
axes[0].set_ylabel(r"fitted  $\hat{\beta}(x)$ ")
axes[0].set_title(f"recovery, corr = {corr:.3f}")

order = np.argsort(test["X2"].to_numpy())
axes[1].plot(test["X2"].to_numpy()[order], beta_true[order], "k-", lw=2, label="true")
axes[1].scatter(test["X2"], beta_hat, s=6, alpha=0.3, color="C0", label="fitted")
axes[1].set_xlabel("$X_2$")
axes[1].set_ylabel(r"$\beta(x)$")
axes[1].set_title(r"$\beta(x)$ against a modifier")
axes[1].legend()
fig.tight_layout()
plt.show()

corr(beta_hat, beta_true) = 0.996
beta0 = -0.960 (true constant part -1.0)
mean |error| = 0.098
```

```
/tmp/ipykernel_3784/2710869176.py:4: UserWarning: Converting a tensor with requires_grad=True to a s
Consider using tensor.detach() first. (Triggered internally at /__w/pytorch/pytorch/torch/csrc/aut
    beta0 = float(flow.nodes["Y"].shifts["T"].beta0)
```

The scatter against X2 alone is a band rather than a line, because the true effect also depends on X3 — exactly as it should be.

5. The read-out is an identity, not a summary

For a binary treatment, $\beta(x)$ equals the difference of the abducted latents between the two arms, with the outcome held fixed. That is a definition the implementation must satisfy, and it does, to float precision:

```
u1 = flow.abduct(test.assign(T=1.0), seed=0)["Y"].to_numpy()
u0 = flow.abduct(test.assign(T=0.0), seed=0)["Y"].to_numpy()
```

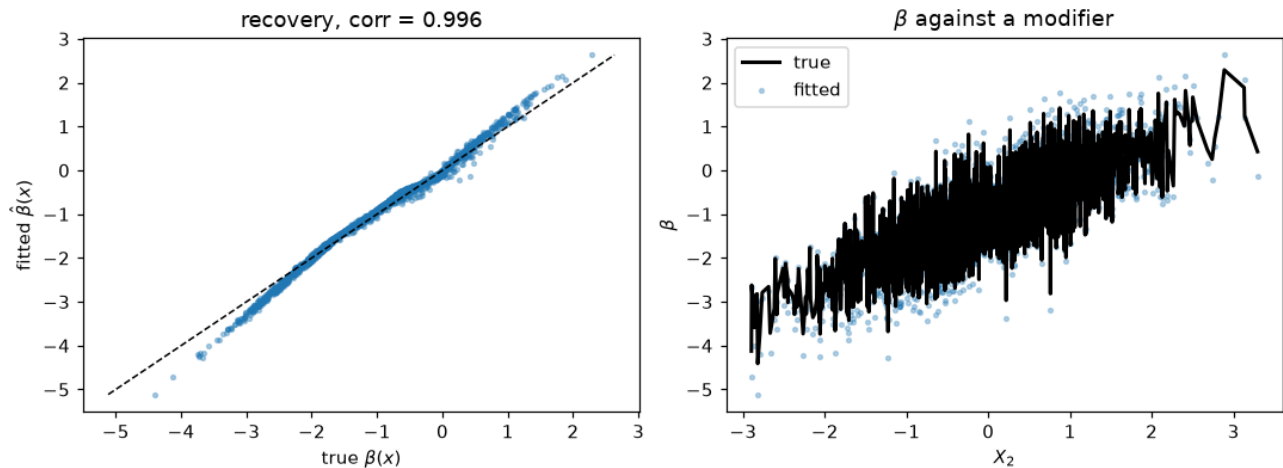


Figure 11: png

```
print(f"max |beta(x) - (u(T=1) - u(T=0))| = {np.abs(beta_hat - (u1 - u0)).max():.2e}")
max |beta(x) - (u(T=1) - u(T=0))| = 4.77e-07
```

6. Confounding: why center= exists

Everything above had a prognostic part flexible enough to absorb g . Real models are misspecified, and when the misfit correlates with **treatment assignment**, the effect head absorbs the confounding instead of the effect.

The configuration where this bites hardest is deliberately simple: one covariate, a strong propensity $e(x) = \sigma(2x)$, a constant true effect $\tau = -1$, and a quadratic prognostic part fitted with a linear term. `center="ps"` replaces t by $t - \hat{e}(x)$ with out-of-fold propensities from the training-frame column `ps`; the guide says why they must be out of fold. Stage 1, the propensities, is yours. Here, five classical fits of the treatment spec.

TAU = -1.0

```
def confounded(n, seed):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    t = (rng.logistic(size=n) > -2.0 * x).astype(float) # strong confounding
    y = (rng.logistic(size=n) - 1.2 * x**2 - TAU * t) / 2.0 # quadratic prognostic
    return pd.DataFrame({"X": x, "T": t, "Y": y})
```

```
c_train, c_val, c_test = confounded(5400, 0), confounded(600, 7), confounded(3000, 1000)
```

```
# stage 1 of the centered design, the caller's job: cross-fitted P(T=1|X), each
# fold predicted by a treatment model that never saw it (the DML requirement)
fold_id = np.random.default_rng(0).permutation(len(c_train)) % 5
e_oof = np.empty(len(c_train))
for j in range(5):
    t_spec = {
        "X": ContinuousNode(I(transform="affine")),
        "T": OrdinalNode(2, LS("X")),
    }
```

```

proxy = CausalFlowDAG(t_spec, seed=0)
proxy.fit_classical(c_train.iloc[:, fold_id != j])
e_oof[fold_id == j] = proxy.pmf(c_train.iloc[:, fold_id == j], "T")[:, 1]

mae = {}
for center in (None, "ps"):
    spec_c = {
        "X": ContinuousNode(I(transform="affine")),
        "T": OrdinalNode(2, LS("X")),
        # linear prognostic term, though the truth is quadratic
        "Y": ContinuousNode(LS("X") + VC("X", center=center, t="T")),
    }
    fc = CausalFlowDAG(spec_c, seed=0)
    fc.fit(
        # the out-of-fold propensities ride the frame as the column center= names
        c_train.assign(ps=e_oof) if center else c_train,
        epochs=250,
        learning_rate=1e-2,
        batch_size=512,
        validation_data=c_val,
        seed=0,
        callbacks=EarlyStopping(), # keep the best-validation weights
    )
    b = fc.varying_coef(c_test, "Y")
    mae[bool(center)] = float(np.abs(b - TAU).mean())
    print(
        f"center={bool(center)!s:5s} mean |beta - tau| = {mae[bool(center)]:.3f}"
        f"    mean beta = {b.mean():+.3f} (true tau {TAU:+.1f})"
    )
ratio = mae[False] / mae[True]
print(f"centering cuts the bias by a factor of {ratio:.1f}")
assert ratio >= 2, f"centering should at least halve the bias: {ratio:.2f}"

center=False mean |beta - tau| = 1.284 mean beta = -0.028 (true tau -1.0)

center=True mean |beta - tau| = 0.308 mean beta = -0.692 (true tau -1.0)
centering cuts the bias by a factor of 4.2

```

Without centering the confounding swallows most of the effect; with centering the estimate returns near the truth. The printed factor is the measured bias reduction, and `tests/test_vc_centered.py` requires at least two.

What to take away

you want	use	read it with
one interpretable effect	<code>LS("T")</code>	<code>ls_coefficients()</code>
an effect that varies with covariates	<code>VC(*modifiers, t="T")</code>	<code>varying_coef(df, node)</code>
the same under confounding + a misspecified prognostic part	<code>VC(..., center="ps")</code>	the same read-out
a shortlist of modifiers before you commit	<code>effect_modifier_scan</code> on a cheap all- <code>ls</code> fit	its flag column, read as screening

Why not a joint CS over treatment and modifiers, and what the penalty and the centering mean:

docs/varying-coefficients.md. The scan: docs/scores.md.